Benchmarking Rule-Based, Classical Machine Learning, and Deep Learning Anomaly Detectors in ETL Data Quality Pipelines

Theo Bigand
ESILV — M2 Data & Artificial Intelligence
Léonard de Vinci Pôle Universitaire
Courbevoie, France
theobigand@gmail.com

***Abstract*—Modern data pipelines depend on automated data quality controls, yet Extract–Transform–Load (ETL) practitioners still rely mostly on hand-written declarative rules. Recent work puts machine learning (ML) anomaly detectors inside ETL workflows, but fair comparisons between rule-based, classical ML and neural detectors, inside a realistic pipeline and broken down by the type of anomaly, remain scarce. I ask which family of detectors to use for which kind of data quality problem, how much the answer depends on the operating point and on time, and what it costs. To answer, I built an open-source ETL pipeline in which seventeen detectors share one `fit/predict/score` interface: three rules (Z-score, IQR, Great Expectations), eight classical ML detectors (Isolation Forest, LOF, One-Class SVM, ECOD, COPOD, HBOS, KNN, PCA) and six neural ones (autoencoder, VAE, Deep SVDD, DIF, AE1SVM, LUNAR). I evaluate them over five seeds on a stratified held-out split, on a pool of 152 open datasets (used as 100 labelled problems and as 42 UCI tables, 58 with ADBench, into which six anomaly types are injected) and on Faker-generated data with controlled anomaly types, and I report each study against hypotheses written before its runs. The aggregate findings on the 100 labelled problems remain descriptive: their historical schema and row filtering precede the split and require train-only revalidation. Under that preparation, the column-wise ECOD, COPOD and HBOS match Isolation Forest, and the neural models do not beat the best classical one. The per-type view shows a narrow, largely non-overlapping competence for each family, which holds in structure on real tables but not in its values. The operating point moves precision and recall by an order of magnitude with the same scores, a threshold fixed on the past fails when the prevalence moves, and the neural models cost one to two orders of magnitude more than the column-wise ones. Broad coverage therefore needs several families, chosen from the type of fault, with the operating point and the prevalence monitored.**

***Index Terms*—data quality, ETL, anomaly detection, benchmarking, machine learning, Great Expectations, Isolation Forest, autoencoder**

I. Introduction

Extract–Transform–Load (ETL) pipelines are the backbone of modern data infrastructure: they move raw operational data into analytical stores where it feeds dashboards, reports, and machine-learning applications. Organizations increasingly manage data as an asset that must be governed [1], and the value delivered downstream is bounded by the quality of the data that reaches the destination. This is why data quality has long been a first-class concern for data engineers, and why automated, continuous quality control is a capability rather than an option.

The dominant approach in industry remains rule-based: practitioners hand-write declarative expectations—"column `amount` must be positive", "column `email` must match a regex"—in frameworks such as Great Expectations, Deequ, or Soda [2]. These rules are auditable and cheap to run, but they scale poorly with schema complexity, detect only violations the author anticipated, and miss subtle multi-column or distributional anomalies. To address these limits, a recent line of work has proposed to complement or replace such rules with machine learning anomaly detectors embedded directly inside the ETL flow [3]–[5]. The existing literature, however, suffers from three gaps.

First, the pipelines I reviewed are mostly presented as architectures. Gopa [3] reports aggregate gains taken from other implementations and releases no code, and the abstract of Marupaka and Rangineni [4] announces accuracy gains without naming a dataset or a baseline. When an experiment is reported, it exercises a single algorithm on a single dataset: Sokkula and Vuppala [5] run Isolation Forest on Credit Card Fraud after oversampling the training data with SMOTE, and report an ROC-AUC of 0.51 with a fraud recall of 0.02. A near-random score on one dataset offers limited insight into when ML-based quality control is actually worth adopting.

Second, the anomaly detection community has produced excellent generic benchmarks—ADBench [6] compares 30 algorithms on 57 tabular datasets; PyOD [7] provides a unified implementation of a wide range of detectors—but these benchmarks evaluate algorithms on their own, outside of an ETL context. ADBench reports training and inference times for each algorithm, but it has no declarative rule-based baseline of the kind that dominates production systems, and it does not measure what a detector costs inside a running pipeline (latency, memory, operational overhead).

Third, benchmarks typically report a single aggregated score per dataset, hiding the fact that different families of detectors excel at different kinds of anomalies [8]. ADBench does look at anomaly types, but as geometric types generated on synthetic data (local, global, dependency and clustered anomalies), not as the data quality faults an ETL pipeline meets. A Z-score detector is essentially unable to flag categorical format violations; Great Expectations trivially catches null-rate spikes but ignores multivariate outliers; autoencoders learn joint distributions but can be fooled by concept drift. A practitioner choosing a detector today lacks a fine-grained, actionable map from anomaly type to recommended method.

a) Research questions.: Each gap becomes a question that the rest of the paper answers, one study at a time.

RQ1 Inside one ETL pipeline and behind one interface, how do rule-based, classical machine-learning and neural detectors compare on real and on synthetic data, and does a larger registry change the answer? (first gap)

RQ2 Which anomaly types does each family detect, and does that per-type profile survive a change of generator, of injection magnitude and a move to real tables? (third gap)

RQ3 How much do the conclusions depend on the operating point (contamination, a label-free threshold) and on time (a chronological split, a change of prevalence)?

RQ4 What does a detector cost inside the pipeline, in time and in memory? (second gap)

Each study has a protocol and hypotheses written in the repository before its runs (Section III-H). A hypothesis that fails is reported as failed. The Discussion answers the four questions in turn.

b) Contributions.: This paper addresses these three gaps. I present:

1) **A reproducible ETL pipeline** in which rule-based and classical ML anomaly detectors share one interface and are scored on the same extract–transform–detect path, released as open-source code. The registry holds three rule-based detectors, eight classical models and six neural ones, all behind the same interface.
2) **A head-to-head benchmark** of statistical rules (Z-score, IQR, Great Expectations), classical ML detectors (Isolation Forest, LOF, One-Class SVM, ECOD, COPOD, HBOS, KNN, PCA) and neural detectors (autoencoder, VAE, Deep SVDD, DIF, AE1SVM, LUNAR) on real and synthetic datasets, with a stratified held-out split, five seeds, and the ML operating point read from the data, reporting precision, recall, F1, ROC-AUC, AP, precision-recall curves, and detection-stage latency (fit, predict and score; extraction, transformation and loading are not timed).
3) **A per-anomaly-type analysis.** Using a synthetic dataset in which six distinct anomaly categories (missing values, duplicates, univariate outliers, multivariate outliers, temporal drift, invalid formats) are injected at a tunable severity, I produce a method-by-type recall and precision matrix, test its sensitivity under a severity sweep, add the neural models and the further classical detectors, and repeat it on two further schemas. The families turn out to have narrow, largely non-overlapping competence that the aggregate metrics hide.
4) **The same analysis on real data.** The six types are injected into 42 real UCI tables of a pool of 152 open datasets, and the detectors are also run on its 100 labelled problems. The synthetic pattern holds only in part, and no detector is clearly ahead on the labelled problems.

The rest of the paper is organized as follows. Section II reviews related work on data quality in ETL and on anomaly detection benchmarks. Section III describes my pipeline architecture, the detectors I compare, the datasets, the metrics, the studies with their hypotheses and the verification of the code. Section IV reports the results, one subsection group per research question: the detectors compared on examples and on the labelled pool, the per-type profile on synthetic and real tables, the operating point in time, the cost, and the extension of every study to the whole pool. Section V answers the research questions, says when rules, classical ML and deep learning are each the right tool and what limits the evidence, and Section VI concludes. Appendix A and B describe the verification of the code and its outcomes, Appendix C gives the full per-type matrices and Appendix D the detailed tables of each study.

II. Related Work

a) Data quality in ETL pipelines.: Data quality is usually framed as governance: Khatri and Brown [1] treat data as an asset to be governed, and recent surveys list the tools that check it, including Great Expectations [9], Deequ and Soda among 17 tools [2], next to a broader survey of pipeline tools [10]. These tools are rule-based. Deequ expresses constraints as unit tests for data that run as aggregation queries on Spark, supports incremental validation, and also suggests constraints and detects anomalies in the time series of quality metrics [11]. They check what the author of the rules thought of in advance; they do not say how a learned detector would have done on the same data.

b) Anomaly detection algorithms.: The detectors I compare are standard. Isolation Forest [12], LOF [13] and the One-Class SVM [14] are available in PyOD [7], and Chandola et al. [15] survey the field. Goldstein and Uchida [8] compare 19 unsupervised algorithms on 10 datasets and find that the choice depends on the type of anomaly, local or global, which is the observation this paper takes to data quality faults. For reconstruction models, Malhotra et al. [16] train an LSTM encoder-decoder on normal behaviour and flag high reconstruction error, on public time series (power demand, space shuttle, ECG) and on engine sensors; my LSTM autoencoder follows that idea on windows of tabular rows.

c) Benchmarks.: ADBench [6] is the closest work: 30 algorithms on 57 datasets, 98,436 experiments, with different levels of supervision and four types of anomalies (local,

global, dependency and clustered) generated from existing data, reported with AUCROC and AUCPR and with training and inference time. In its text I found no threshold-dependent metric and no chronological protocol, and none of its anomaly types is a data-quality fault such as a missing value or an invalid format. Agyemang [17] compares five unsupervised detectors on one simulated dataset with accuracy, precision, recall and F_1, and leaves real data as future work (I read the abstract only). AnoShift [18] builds a time-ordered protocol on ten years of Kyoto-2006+ network traffic, training on 2006–2010 and testing on 2011–2013 (NEAR) and 2014–2015 (FAR), and reports ROC-AUC and AP for classical and deep models, again without a decision threshold. FedAD-Bench [19] compares deep models in a federated setting on four tabular datasets and chooses each threshold by maximising F_1 on a validation set with the anomaly ratio of the test set. Sun et al. [20] propose online thresholds with confidence sequences that adapt to distribution shift, and test them on synthetic and real data. They study how to pick a threshold, not how classical detectors behave in a pipeline.

d) Machine learning in ETL pipelines.: Papers on machine learning inside ETL mostly describe architectures. Gopa [3] reports aggregate figures taken from enterprise deployments without code, and Marupaka and Rangineni [4] announce accuracy gains in an abstract that names no dataset or baseline. Sokkula and Vuppala [5] run an Isolation Forest on the resampled Credit Card data and obtain an AUC-ROC of 0.51.

e) Gap and what is not claimed.: Among these sources I did not find a comparison of rule-based, classical and learned detectors that reports their decisions, not only a ranking, broken down by the kind of data-quality fault, on both a controlled generator and real tables, and in the setting of an ETL pipeline with its cost. That is the gap this paper addresses, with the limits stated in Section V. I do not claim that no such study exists: the scan covers the works above, was not systematic, and I read the full text of only some of them. I do not claim that the detectors are new, that the injected faults match natural ones, or that the results hold beyond the datasets used.

III. Methods

A. Pipeline

Detectors are evaluated inside a minimal Extract–Transform–Load pipeline rather than in isolation. The extract stage reads the raw table, the transform stage removes exact duplicate rows, and the quality stage applies a detector that assigns each row a boolean anomaly flag and a continuous anomaly score. All detectors share a common `fit`/`predict`/`score` interface modelled on PyOD [7], so that classical and neural models can be dropped in without changing the harness. The ordinary transform removes exact duplicate rows before detection. This is an operational cleaning decision, not a detector result: in the typed injection study, duplicates are instead retained after injection and a separate `transform_dedup` reference is reported. That study has its own group-preserving split and does not reuse the ordinary ETL path.

B. Rule-based detectors

This stage compares three generic rule-based baselines. Their numerical bounds are fitted on training data; a row is flagged if it violates any active column rule. These baselines do not represent the full range of domain-informed declarative constraints.

For *Z-score*, let $\hat{\mu}_j$ and $\hat{\sigma}_j$ be the training mean and sample standard deviation of numeric column j. The row score is

$$s_Z(x) = \max_j \frac{|x_j - \hat{\mu}_j|}{\hat{\sigma}_j}, \qquad \text{flag if } s_Z(x) > 3. \tag{1}$$

Constant training columns have zero contribution and do not trigger flags. For a normal variable with known population parameters, about 99.73% of values lie within 3σ. This is a per-column reference, not a calibrated row-level false-alarm guarantee for fitted, multivariate data. Extreme training values can inflate the fitted mean and standard deviation and mask other outliers.

IQR fits Q_{1j} and Q_{3j} on each numeric training column and sets $I_j = Q_{3j} - Q_{1j}$. Its fences are

$$L_j = Q_{1j} - 1.5I_j, \qquad U_j = Q_{3j} + 1.5I_j. \tag{2}$$

The row is flagged if any active column lies strictly outside its fences; the score is $\max_j \max(0, L_j - x_j, x_j - U_j)/I_j$. Columns with zero training IQR contribute zero and do not trigger flags in this implementation. Quartiles are less sensitive to extreme values than the mean and standard deviation, but training contamination can still move them. Under a normal distribution the population fences lie at approximately $\mu \pm 2.7\sigma$, giving a two-sided per-column tail of about 0.70%, versus about 0.27% for the 3σ rule. With d independent normal columns and known population bounds, a per-column tail probability p gives row alert probability $1 - (1-p)^d$; dependence and fitted bounds change that reference. Neither rule fixes a universal row alert rate.

Great Expectations implements a generic automated statistical suite [2]: a not-null check on every column and an inclusive range check on each numeric column, using its training 0.1% and 99.9% quantiles. A range check is omitted if either bound is undefined or the bounds coincide. A row is flagged when it violates at least one check; its score is the fraction of active checks violated. The fitted percentile bounds are univariate statistical detectors expressed as declarative expectations. They require no Gaussian model, but they do learn marginal limits from the training distribution. The suite contains no domain-specific validity, cross-field consistency, category or format constraints; its results must not be generalized to such richer rule suites.

C. Classical machine-learning detectors

The second family replaces the per-column band with a learned model of the data, so that a row can be flagged for an implausible *combination* of otherwise-normal values.

TABLE I
WHAT EACH WORK REPORTS, AS FAR AS I COULD READ IN THE CITED SOURCES. "NOT FOUND" MEANS ABSENT FROM THE TEXT I READ, NOT PROVEN ABSENT.

Work	Detectors and data	Time order	Metric at a threshold	Per-type results
ADBench [6]	30 algorithms, 57 datasets	not found	not found (AUCROC, AUCPR)	four geometric types, generated
Agyemang [17]	5 detectors, one simulated set	none	accuracy, precision, recall, F_1	not stated in the abstract
AnoShift [18]	classical and deep, Kyoto-2006+	IID, NEAR, FAR splits	not found (ROC-AUC, AP)	none
FedAD-Bench [19]	deep models, federated, 4 datasets	not found	F_1-optimal threshold on a validation set	none
Sun et al. [20]	thresholding method, synthetic and real	shift in the stream	error-rate guarantees	none
This paper	6 detectors (and 2 deep), 3 real sets, a pool of 152, synthetic and typed injection	chronological pilot and decision study on two datasets	precision, recall, F_1 at the training-prevalence threshold	six data-quality fault types

Eight unsupervised detectors are taken from PyOD [7] rather than reimplemented, with two deviations explained below. All receive the same preparation: non-numeric columns are dropped, missing cells are median-imputed, and every feature is standardized, since several of them are distance-based. The PyOD `decision_function` provides the continuous score; the boolean flag uses a *contamination* equal to the anomaly prevalence of the training split (clamped to PyOD's valid range), so each model marks about as many rows as the data actually contains rather than a fixed share. Contamination sets only the flag threshold and leaves ROC-AUC and AP unaffected. Base estimator defaults are retained unless the recorded protocol or an adaptation below overrides them; no hyperparameter search was performed. This gives the learned models numeric information only. Great Expectations also checks nulls in every column, whereas neither path evaluates text, categorical-domain or cross-field rules; this unequal information access is a limitation of the comparison rather than a family ranking.

Three of the eight are the usual references. *Isolation Forest* [12] scores a row by how few random axis-aligned splits are needed to isolate it; *Local Outlier Factor* (LOF) [13] compares a point's local density to that of its neighbours; and a *One-Class SVM* [14] fits a boundary enclosing the bulk of the training data.

Five more complete the family. ECOD, COPOD and HBOS aggregate marginal evidence without explicitly fitting cross-feature dependence; several moderate marginal deviations can nevertheless accumulate into a high row score. *COPOD* [21] is motivated by copulas, which separate marginal distributions from dependence. This motivation must be distinguished from the additive marginal-tail calculation in the evaluated implementation; it does not fit a general joint empirical copula.

The inspected reference is PyOD 3.6.7. Write $u_{Lj} = -\log p_{Lj}$ and $u_{Rj} = -\log p_{Rj}$ for column j's left and right empirical tail surprisals. Its skew-selected term u_{Sj} is u_{Lj} for negative skewness, u_{Rj} for positive skewness, and $u_{Lj} + u_{Rj}$ for zero skewness. The column contributions, summed over j, are

$$c_j^{\mathrm{ECOD}} = \max(u_{Lj}, u_{Rj}, u_{Sj}),$$
$$c_j^{\mathrm{COPOD}} = \max\big(u_{Sj}, (u_{Lj} + u_{Rj})/2\big).$$

Thus *ECOD* [22] does not merely select one tail, and COPOD does not merely average two tail probabilities: it averages their negative logarithms and compares that average with the skew-selected term. *HBOS* [23] builds a histogram of each column and scores a row by the sum of the logarithms of the inverse heights of the bins its values fall in. The two others use the joint distribution. *KNN* [24] scores a row by its distance to its fifth nearest neighbour among the fitted rows, the simplest distance-based detector and the one LOF refines. *PCA* projects the rows onto the principal components that explain 95% of the variance and scores the squared reconstruction error: a row that follows the correlations of the fitted data is rebuilt almost exactly, one that breaks them is not, while a shift that stays inside the kept components is invisible to it. The idea of detecting outliers with principal components is that of Shyu et al. [25].

The ETL row-level contract requires a fitted model to score a row without using other evaluation rows. PyOD 3.6.7 recomputes both tail distributions and skewness on training rows concatenated with the evaluated batch. The repository's inductive ECOD/COPOD variants instead freeze sorted training columns and training skew signs. For n training values, they use $p_{Lj}(x) = (1 + \#\{i : X_{ij} \le x_j\})/(n+1)$ and the analogous $p_{Rj}(x)$ with $X_{ij} \ge x_j$, counting ties in both tails. These ranks match inserting the query alone into the reference set, but the frozen skew sign need not match PyOD's recomputed sign. General singleton-score equivalence is therefore not claimed; training-score thresholds can also differ from upstream. The existing registry tests compare each of the 17 row-level detectors scored alone and in a batch, with float32 tolerances for neural models. They provide bounded contract evidence, not proof for every dataset or configuration.

These are algorithm-changing protocol adaptations, not corrections that establish equivalence to unmodified PyOD. Their impact requires the separate ablation in the remediation

roadmap; historical outcomes are retained under the settings actually evaluated. PyOD's own PCA scores a weighted distance to the eigenvectors, not a reconstruction error, so the projection comes from scikit-learn. LOF and KNN compare every row with the fitted rows, and the One-Class SVM is quadratic too; they are skipped, and the skip recorded, above 50,000 fitted rows for LOF and KNN and 20,000 for the One-Class SVM (limits set from the cost study).

D. Deep detectors

The third family learns a representation with a neural network. They are opt-in in the registry, they need PyTorch, and they run on the CPU. Six models are compared behind the same interface: one scikit-learn MLP autoencoder and five PyOD 3.6.7 models [26], with the protocol adaptations described below. Defaults are retained only where the recorded configuration does not override them. The complete implementation inventory for all seventeen comparable detectors—family, pinned implementation origin, accepted input, training-only preparation, training cap, score direction, decision rule and adaptation—is maintained with the source in `verification/detector_inventory.md`. It records the protocol; it is not an empirical comparison or a convergence result. The *autoencoder* is a multilayer perceptron (scikit-learn) trained to rebuild the standardized features through a bottleneck; a row that breaks the structure learned on the fitted data is rebuilt badly, and the squared reconstruction error is its score. The *variational autoencoder* (VAE) [27] encodes a row into a Gaussian code and decodes it, and is trained on a bound of the likelihood of the data; its score is the reconstruction error. PyOD draws the code at random when it scores, so a row's score changes from one call to the next and with the rows around it; I decode the latent mean instead and use the Euclidean reconstruction distance. This changes the scoring rule; it is not the expected reconstruction error over latent draws. *Deep SVDD* [28] trains a network to map the fitted rows close to a fixed centre and scores a row by its distance to that centre. The *Deep Isolation Forest* (DIF) [29] maps the rows through an ensemble of randomly initialized networks and builds an isolation forest in each representation; PyOD standardizes the representation of the scored rows with their own mean and variance, which makes a score depend on the other scored rows, so I fit one representation scaler per network on the training rows and reuse it, followed by the same hyperbolic tangent transform. *AE1SVM* [30] trains an autoencoder and a one-class SVM approximated with random Fourier features on its code, and scores the combination. The adapter duplicates the last row when a training or scoring pass would end in a singleton batch; the extra score is then removed. Training padding changes that row's weight and is not an inference-only repair. *LUNAR* [31] learns the score from the distances of a row to its nearest neighbours, which generalizes LOF and KNN with a network. The neural models are fitted on at most 20,000 randomly drawn fitted rows to bound the duration, and each is seeded with the seed of the repetition. The LSTM autoencoder of Appendix D-J reads windows of ordered rows, does not share the row-level contract, and stays outside this registry.

E. Synthetic per-anomaly-type dataset

The real datasets carry a single binary label that mixes every kind of defect together, so they cannot say *which* problem a detector is good at. I therefore generate a controlled dataset of 12,360 customer records with Faker and NumPy, in which income is correlated with balance and transaction size and account age is spread over three years. Into this clean population I inject six anomaly types, each at a rate of 3% and each row tagged with its category: *missing values* (one or two cells nulled), *exact duplicates* (a clean row copied), *univariate outliers* (one numeric field pushed far out of range), *multivariate outliers* (every field individually in range but the combination implausible, e.g. an 18-year-old with a top-percentile balance and income), *temporal drift* (recent-signup rows whose income and balance distribution has shifted upward by half), and *invalid formats* (malformed e-mail, out-of-domain country code, impossible age). Because the category of every injected row is known, I report, per method and per type, both the detection rate (recall) and the precision on a stream made of clean rows plus that one type. The injection magnitudes are exposed as a single *severity* multiplier so the choice of magnitude can be swept rather than fixed (Section IV-D). I also add a time-aware detector, *drift*, that learns a per-feature baseline from the oldest rows and flags recent rows that both sit in a window whose mean has moved and lean in the direction of that move; it is run on the synthetic dataset only, as the other detectors ignore the row order.

F. Datasets

The real data is one pool of 152 open datasets from ADBench and the UCI repository (open licence, pinned by checksum; `verification/dataset_pool.md`). I use it in two ways: 100 of its problems already carry labelled anomalies and are run as they are (Section IV-B), and 58 of its tables are large enough (3,000 rows, four continuous columns) to receive injected anomalies: the 42 UCI ones form the typed-injection study (Section IV-F) and the 16 ADBench ones join them in the whole-pool study (Section IV-K). Three datasets are worked out in detail as examples, because they illustrate three regimes of the method. *Credit Card Fraud* [32] (Kaggle, also one of the ADBench problems; 283,726 transactions after de-duplication, 30 numeric features, 0.17% labelled fraud) is the standard strongly imbalanced benchmark, but its positive class mixes every kind of defect. *Cardio* (a cardiotocography anomaly dataset as packaged for outlier-detection benchmarking, another ADBench problem; 1,831 rows, 21 standardized features, 9.6% anomalies) is a *genuinely imbalanced* real dataset [6], [33]: the anomalies are rare and distributionally distinct, the regime the unsupervised detectors are built for. *Water Quality* [34] (a Kaggle dataset of the same kind, outside the pool; 3,276 samples, 9 numeric features) is the opposite case: the target class ("not potable")

is common (61%) rather than rare, and the data has genuine missing values, so it probes the detectors outside their regime. Beside the real data, a *synthetic* dataset generated with Faker ($12{,}360$ rows, 17.5% anomalous, described above) carries the per-type labels, and two further generators repeat it on other schemas (Appendix D-E), to test particular cases in controlled numbers.

G. Evaluation protocol

Each dataset is split $70/15/15$ (train/validation/test), stratified on the label. For the synthetic dataset, rows are grouped by customer identity before stratification so an original and its exact copy cannot cross partitions. Detectors are fitted on the train split and every reported number is computed on the held-out test split; the validation split is reserved for later tuning. The ML contamination is read from the train split's prevalence. This is a label-informed operating-point choice, although estimator fitting itself does not use the labels. The real-data benchmark also permits a contamination value specified before evaluation, independent of every split's labels. This label-free preset forms a distinct comparison protocol; the numerical results in this manuscript retain the train-prevalence baseline. In the sensitivity sweep, contamination follows a fixed grid and no value is selected from the test metrics. Threshold or hyperparameter selection for a later confirmatory experiment would use training or validation data only. The row identifiers of the historical pool splits were not archived when the runs were made. I recomputed them from the deterministic split and the recorded seeds, and for all 100 labelled problems a replay of the Z-score detector on the recomputed partitions reproduces the archived per-seed metrics to 10^{-9}; the partition digests and sizes are published with the evidence. This confirms the training and test rows used by that deterministic detector, not the validation rows, which no reported number uses. The whole procedure is repeated over five random seeds and I report the mean and standard deviation. This spread measures the sensitivity to the split and to model randomness on a fixed dataset; it is not a confidence interval for performance on new data. LOF and the One-Class SVM are run on Water Quality, Cardio and the synthetic dataset but not on Credit Card Fraud, where LOF's superlinear scoring and the One-Class SVM's quadratic training were expected to be too costly at 284k rows; this was not measured.

Because the detectors emit both a threshold decision and a graded score, I distinguish ranking metrics from threshold-dependent metrics. Average Precision (AP) and ROC-AUC use continuous scores, with larger scores indicating more anomalous rows; precision, recall, F_1 and the fraction of rows flagged use the selected decision threshold. Changing that threshold alone leaves the ranking metrics unchanged.

The primary ranking metric is non-interpolated AP, computed with `average_precision_score` in scikit-learn. At the successive distinct score cutoffs, ordered from the highest score to the lowest, let P_j and R_j denote precision and recall, with $R_0 = 0$. Then

$$\mathrm{AP} = \sum_{j=1}^{J} (R_j - R_{j-1}) P_j. \tag{3}$$

Tied scores enter together at one cutoff. This recall-weighted sum does not linearly interpolate precision and differs from trapezoidal area under a precision–recall curve. Historical result files retain the field `pr_auc` for this same AP quantity; displayed AP labels in the results refer to AP, not to a second metric.

For rare anomalies, AP makes false alarms visible through precision. ROC-AUC remains complementary: its false-positive rate can be small despite many false alarms when the normal class is large. For m anomalies among n evaluated rows, prevalence $\pi = m/n$ is a no-skill reference precision, not an exact finite-sample expectation of AP for a uniformly random strict ranking. Such rankings have an upward finite-sample correction; constant scores, with all ties evaluated together, give AP exactly π. AP therefore requires the evaluation prevalence for interpretation, and raw AP values across different prevalences are not directly comparable. Neither AP nor ROC-AUC alone establishes the usefulness of a deployed operating point. I also report detection latency. Because a single flag threshold hides the rest of the trade-off, I additionally draw the full precision–recall curve of every detector (Fig. 4) and sweep Isolation Forest's contamination across each dataset (Table IV).

a) Historical pool preparation and exploratory correction.: The labelled-pool loader filters columns by missingness, constancy and category count, constructs the one-hot vocabulary and removes duplicates with conflicting labels before splitting. Median imputation and numerical scaling are fitted on the training partition, but the preceding schema and population decisions can read evaluation values or labels. A constructed six-row table changes from five to three encoded columns when only two evaluation categories change; changing an evaluation duplicate's label can also remove a reference row. These checks establish an information-access problem, not its effect on dataset-level AP.

An exploratory `pool-train-only-v1` path instead requires explicit, source-bound train/validation/test positions. It fits column selection, vocabulary, medians and scaling on training rows, retains the supplied rows and order, rejects raw-feature duplicates across partitions and encodes unseen categories without extending the training vocabulary. Labels are excluded from preparation; only training labels calibrate the historical prevalence-based operating point. Constructed regression and defect-injection checks exercise this path. It has not replaced the historical pool studies: source-bound partition recovery and replay are still required. The archived protocols, assignments and reported numbers are retained, and no corrected pool ranking is claimed here.

H. Studies, research questions and hypotheses

Every study below answers one of the research questions of Section I-0a, and its protocol and hypotheses were committed to the repository before its runs (the file is named for each one). A hypothesis that fails is reported as failed, and a choice made after seeing a result is labelled as exploratory.

a) Typed injection on real tables (RQ2).: The six anomaly types were injected into 42 real tables from the UCI repository (open licence, pinned by checksum; `verification/dataset_pool.md`); 58 pool tables are large enough (3,000 rows, four continuous columns, ten distinct values), the 16 other ones come from ADBench and enter in the whole-pool study below. For each table only the continuous columns are kept, and for labelled sets only the normal class, so that the original anomalies never enter the evaluation. The injection follows the synthetic one (three percent of rows per type, same split, seeds and detector settings), with numeric versions of the types: a value several standard deviations away, a pair of correlated columns pushed to opposite tails, a shift of three columns for the most recent rows, a sign flip, a sentinel or a thousandfold scale error. Five hypotheses (`verification/typed_injection.md`) concern the structural rows, the multivariate gap, the specialisation of the types and the stability of the matrices between tables.

b) Further schemas and neural rows (RQ1, RQ2).: Two more generators, a table of card transactions and a table of sensor readings, each with six numeric fields, a date, a string with a checkable format and a category with a finite domain, receive the same six types at the same rate and the same protocol, with four hypotheses (`verification/multi_schema.md`). The sklearn autoencoder and an LSTM autoencoder were added to the synthetic matrix on the same identity-grouped splits and seeds, trained until their validation loss stopped improving (patience of 5, at most 300 epochs), on windows of the ten preceding customers ordered by signup date for the LSTM, with four hypotheses (`verification/deep_typed.md`). The ten detectors added to the registry were then run on the same matrices and on the other studies, with six hypotheses (`verification/full_registry.md`).

c) Labelled problems of the pool (RQ1).: Among the 152 pool tables, 100 problems already carry labelled anomalies: 36 from ADBench and 64 from the UCI repository, where the rarest class (at least 20 rows) is the anomaly and the other classes are normal; a class above 10% of the rows is resampled to 10%. Every table gets one normalisation, a stratified 70/15/15 split, five seeds, a contamination equal to the training prevalence and default settings (`verification/labelled_pool.md`). Neural models are fitted on at most 20,000 rows, and LOF, KNN and the One-Class SVM are skipped above 50,000, 50,000 and 20,000 training rows; every skip is recorded.

d) The whole pool (RQ1 to RQ4).: The studies that were first run on three examples and on the synthetic data were extended to every pool dataset for which they are defined, with six hypotheses (`verification/pool_extension.md`) and the list of the datasets left out and why. (i) One fit per problem, seed and detector gives the metrics at the training-prevalence threshold, the metrics at a label-free contamination of 0.02 (derived from the stored training scores; a test checks that this equals a refit), the precision-recall curve of seed 42 and the contamination sweep of Isolation Forest over a fixed grid. (ii) The severity sweep multiplies the injected amplitude by 0.5, 1, 2 and 4 on every eligible table, with three seeds. (iii) The pipeline runs end to end (read, remove duplicates, detect, write) on the 152 datasets with the Z-score and Isolation Forest, in a fresh process per measure. (iv) The chronological split needs a real time variable: a column readable as a date or timestamp, or a month whose order is checked, not a column whose name only suggests time; at least 3,000 rows remain and the test partition holds at least 20 anomalies. (v) Every local file is compared with the pinned manifest.

e) Chronological pilot (RQ3).: The protocol was written before any score was computed on held-out data (`verification/temporal_pilot.md`). The rows are the cleaned Credit Card rows (Cardio and Water Quality have no time variable), the clock variable is removed from the features, and the chronological split uses the usual 70/15/15 ratios by time. The control uses the stratified split of the main benchmark with the same rows and features, over five seeds. Intervals for the chronological arm come from a block bootstrap over 30-minute blocks of the test period.

f) Decisions under temporal shift (RQ3).: A second study runs on Credit Card and on Bank Marketing, a UCI table (CC BY 4.0) whose file is ordered by date from May 2008 to November 2010, with four hypotheses (`verification/temporal_decision.md`). The Kyoto-2006+ traffic used by AnoShift was left out, because I could not verify its data rights nor download it in time. The date of Bank Marketing is a month, with the file order inside each month. Four detectors (Z-score, IQR, Great Expectations, Isolation Forest) are fitted on the oldest 70% of the rows and scored on the newest 15%. A threshold is a quantile of the scores that keeps the training prevalence, taken (P0) from the training scores, (P1) from the validation scores, or (P2) from the scores of the two previous blocks of the test period, without any label of that period. A stratified control, over five seeds, is also matched to the prevalence of the chronological test.

g) LSTM pilot (RQ1).: Credit Card has no cardholder identifier, so the window is the global stream of the ten previous rows, not a customer's history. The protocol was fixed beforehand (`verification/lstm_pilot.md`). The comparison uses the chronological partitions of the pilot above, the same 50,000 training rows and a CPU budget of at most 20 epochs for both models; the architecture (window of 10 rows, 32 units) was not tuned, and the validation rows served only for early stopping on an unlabelled reconstruction loss.

h) Cost (RQ4).: The protocol was fixed in advance (`verification/cost_study.md`): synthetic Gaussian matrices of 1,000 to 300,000 rows and 8 features, five repetitions up to 30,000 rows and three above, a budget of 300 seconds per measurement, and one fresh process per measurement

so that the peak resident memory, native allocations included, belongs to that run alone. The detector is fitted and applied to the same rows, so this is a cost measurement, not a quality one.

I. Verification and reproducibility

The code that produces the results is checked at several levels, described in Appendix A. A routine suite exercises numerical contracts on small generated tables; a mutation campaign and twenty-three deliberate scientific defects ask whether those tests can fail; a real-data check compares the three research files with an archived manifest; and a release script turns the evidence into a verdict (*certified*, *incomplete* or *rejected*) tied to the source revision and to a fingerprint of the code that computes results. The verdict states where the evidence was produced and never that it was independently attested: I wrote the code and launched every run. A failed or missing check is a limitation of the evidence, not a successful experiment. The measured outcomes are in Appendix B. The chronological split and the window-based drift monitor have a separate protocol: their alert and delay measures are not pooled with the row-level metrics of the stratified benchmark.

IV. Results

The results follow the four research questions. Tables report means over five seeds on the held-out test split (Section III-G) unless said otherwise; seed variability is in the archived result tables, and timing depends on the execution environment. The per-dataset tables and the detailed text of the narrower studies are in Appendix D; the numbers quoted here come from them.

A. Three examples and two more regimes (RQ1)

a) Three regimes.: The three worked examples bracket the method (Tables XII, XIII and XIV of Appendix D). On Credit Card Fraud, where the positive class is a 0.17% label that mixes every defect, the statistical rules recover most frauds (recall 0.87–0.97) at a precision below 0.04. With a contamination matched to the prevalence, the learned detectors flag about 0.2% of the rows: HBOS has the best precision (0.34 ± 0.05), Deep SVDD the best F_1 (0.35 ± 0.08) and Isolation Forest reaches $F_1 = 0.20 \pm 0.07$. On Cardio, where the anomalies are rare (9.6%) and distinct, the learned detectors work: Isolation Forest reaches $F_1 = 0.62$, AP 0.70 and ROC-AUC 0.94, ECOD and COPOD reach an AP of 0.69, DIF and AE1SVM come close, and the Z-score does well too ($F_1 = 0.61$) because the 21 features are already standardised. On Water Quality, where the target class ("not potable") covers 61% of the rows, every ROC-AUC lies between 0.45 and 0.51 and every AP within 0.05 of the base rate: the detectors are at chance, the expected negative result. A learned detector pays off when the anomalies are rare outliers, not when the positive class is a mixed-defect label or a majority.

b) Two more regimes from the pool.: The three examples were picked for the regimes they show, and the pool holds two regimes they do not. Among the 100 problems, the largest margin of the rule-based detectors over the learned ones is on Annthyroid (7.6% anomalies): the IQR rule reaches an AP of 0.61 and the best learned model 0.31. The largest margin of a neural model over the other families is on Mushroom (10% anomalies, a class derived by a rule): LUNAR reaches 0.96 and the best classical model 0.63. I chose them after seeing the results, as illustrations, so this is an exploratory choice and not a test. Table II lists the seventeen detectors on these two problems and on Credit Card Fraud and Cardio as they appear in the pool, normalised like the others, so their numbers differ slightly from the per-dataset tables of Appendix D. It shows the absence of a universal detector at the scale of a single dataset: the IQR rule is the best on Annthyroid and at chance on Mushroom (0.10), where LUNAR leads; Deep SVDD is first on Credit Card Fraud and Isolation Forest on Cardio.

TABLE II
AP of the seventeen detectors on four problems of the pool (mean over five seeds). Credit Card Fraud (0.2% anomalies) and Cardio (9.6%) are the examples of Section IV; Annthyroid and Mushroom are the two additional ones. A dash means the detector was skipped on that size.

Detector	Fraud	Cardio	Annthyroid	Mushroom
Z-score	0.10	0.61	0.18	0.44
IQR	0.02	0.30	**0.61**	0.10
Great Expectations	0.22	0.19	0.10	0.12
Isolation Forest	0.13	**0.70**	0.29	0.63
LOF	–	0.24	0.21	0.26
One-Class SVM	–	0.63	0.19	0.40
ECOD	0.21	0.69	0.28	0.41
COPOD	0.26	0.69	0.18	0.44
HBOS	0.26	0.58	0.24	0.50
KNN	–	0.47	0.23	0.59
PCA	0.03	0.37	0.12	0.37
Autoencoder	0.11	0.35	0.14	0.68
VAE	0.10	0.45	0.18	0.49
Deep SVDD	**0.34**	0.29	0.23	0.40
DIF	0.27	0.67	0.23	0.45
AE1SVM	0.18	0.64	0.31	0.40
LUNAR	0.15	0.33	0.20	**0.96**

B. Labelled problems of the pool (RQ1)

The following tables retain the historical pool results. Because schema and population filtering precede the split, they are descriptive outcomes of that preparation, not a validated strictly train-only comparison. The three examples above are only a few datasets. The pool of 152 open tables (`verification/dataset_pool.md`), of which Credit Card Fraud and Cardio are members, holds 100 problems whose anomalies are already labelled: 36 from ADBench and 64 from the UCI repository. The protocol of Section III-H gives every problem the same normalisation, split, seeds and operating point; the neural models are fitted on at most $20{,}000$ rows. LOF and KNN are skipped above $50{,}000$ training rows and One-Class SVM above $20{,}000$, so they cover 92, 92 and 84 problems; the skips are listed in the archive. The archived runs record no exception for the six neural models on these 100 problems. They do not retain loss histories, completed epochs or convergence diagnostics for the PyOD models, so successful

TABLE III
MEDIAN OVER THE LABELLED PROBLEMS OF THE SEED-AVERAGED TEST METRICS, WITH A 95% BOOTSTRAP INTERVAL OVER THE PROBLEMS, AND THE MEDIAN RANK OF THE METHOD ON AP AMONG THE SEVENTEEN (1 IS BEST). THE FIRST BLOCK IS THE RULES, THE SECOND THE CLASSICAL MODELS, THE THIRD THE NEURAL ONES.

Detector	Problems	AP	ROC-AUC	Rank
Z-score	100	0.203 [0.164, 0.279]	0.626	9.0
IQR	100	0.144 [0.117, 0.191]	0.556	12.0
Great Expectations	100	0.115 [0.105, 0.123]	0.508	16.0
Isolation Forest	100	0.215 [0.158, 0.293]	0.677	7.0
LOF	92	0.165 [0.153, 0.237]	0.588	10.0
One-Class SVM	84	0.240 [0.177, 0.307]	0.670	8.0
ECOD	100	0.240 [0.191, 0.305]	0.656	7.0
COPOD	100	0.216 [0.162, 0.314]	0.691	7.0
HBOS	100	0.237 [0.176, 0.311]	0.663	8.0
KNN	92	0.203 [0.172, 0.284]	0.658	8.0
PCA	100	0.178 [0.152, 0.219]	0.581	10.0
Autoencoder	100	0.205 [0.150, 0.240]	0.631	8.0
VAE	100	0.216 [0.163, 0.279]	0.676	7.0
Deep SVDD	100	0.181 [0.147, 0.226]	0.619	10.0
DIF	100	0.226 [0.173, 0.272]	0.639	8.0
AE1SVM	100	0.210 [0.172, 0.253]	0.635	8.0
LUNAR	100	0.183 [0.153, 0.225]	0.615	10.0

execution is not evidence that those models converged; the limitation is recorded in the remediation audit.

No detector is clearly ahead. ECOD and the One-Class SVM share the highest median AP (0.240), followed by HBOS (0.237) and, among the neural models, DIF (0.226); the intervals overlap broadly with each other and with the Z-score, so the hypothesis that no method is universal is supported on the first six detectors, and it also holds for the seventeen. The best method differs from one problem to the next: among the first six, Isolation Forest is first on 26 problems, LOF on 24, the IQR rule on 16, the Z-score on 14, One-Class SVM on 13 and Great Expectations on 7; among all seventeen, DIF is first on 13 problems, HBOS on 10, the IQR rule on 9, and every method is first on at least two. Learning does help on median: Isolation Forest is above the Z-score on 67 problems and above the IQR rule on 62, but the gain is small (about 0.01 to 0.04 of AP). The median ROC-AUC stays between 0.51 and 0.68, and on 30 problems no detector reaches 0.6. These anomalies are mostly not outlying in the sense these detectors assume, which is expected for labels taken from a rare class rather than from a data-quality defect. Great Expectations, built for declared constraints, is close to chance here.

Three hypotheses of the registry study concern this table. The first holds: ECOD, HBOS and COPOD have a higher median AP than Isolation Forest (0.240, 0.237 and 0.216 against 0.215), although the gap is small and KNN (0.203) and PCA (0.178) are below it. The second holds too: the best neural model (DIF, 0.226) is below the best classical one (0.240), so a neural model does not beat the classical ones by the margin of 0.02 that I had set. The third holds, for the reason given above: no neural model failed.

Two of the five hypotheses on the first six detectors are not supported. The advantage of Isolation Forest over the Z-score was expected to vanish when anomalies are more than 20% of the rows; it is smaller there (+0.013 of median AP on 12 problems) but still positive, as it is below 5% (+0.037, 19 problems) and in between (+0.009, 69 problems). The IQR rule was expected to flag more than three times the anomaly prevalence; its median is 2.3 times, the Z-score's 1.7, and the learned detectors flag about the prevalence by construction. The order of the methods is the same for ADBench and for UCI (rank correlation 0.94), although the level is much lower on UCI (best median AP 0.19 against 0.40). Fifteen sources underlie both an ADBench and a UCI problem (17 and 15 of the problems, with different derivations), so the problems are not independent, and 10% resampling fixes the median prevalence at 0.10. The ADBench half is closer to the usual benchmark and the UCI half is a harder, rule-derived labelling; I did not tune any detector, so the numbers describe default settings.

C. Synthetic data: aggregate and per type (RQ2)

Aggregate metrics on the synthetic data compress the families together. Over the seventeen detectors F_1 ranges from 0.22 (Deep SVDD) to 0.59 (DIF) and AP from 0.22 to 0.65 (Table XV of Appendix D); among the earlier detectors Isolation Forest has the best F_1 (0.50), and Z-score and HBOS reach a perfect precision at a recall of 0.20. Read alone, these numbers suggest that the choice of detector barely matters. The per-type breakdown shows the opposite.

Fig. 1 gives the mean recall of each method of the registry on each injected type, with the LSTM autoencoder of Appendix D-J for reference. The structure the aggregate table hides is now clear: most methods have a narrow competence, and the neural models and the new classical ones change the picture only on some columns.

- *Univariate outliers* are caught by every distributional method (recall 0.68–1.00) except Deep SVDD (0.33), and only by those. Z-score's precision on them is 1.00: it never fires on a clean row.
- *Missing values* are caught best by the Great Expectations not-null expectations (recall 1.00, precision 0.94), because the statistical rules propagate `NaN` to a non-flag and the learned models see imputed values. LUNAR (0.44) and LOF (0.28) are the next best, and every other model stays below 0.20.
- *Exact duplicates* are nearly invisible to every detector (recall at most 0.18, except LUNAR with 0.39). The deduplication rule of the transform stage is the only thing that catches them, at precision 0.50 because it also flags each row's clean original. In this grouped split the rule is evaluated within the held-out partition; it is a stream-level reference, not a model.
- *Multivariate outliers* separate the families, and here the new detectors matter. Isolation Forest (0.89) and the One-Class SVM (0.90) were the only methods with real recall among the first six, since they are the only ones that model the joint distribution. Among the neural models, DIF (1.00), the VAE (0.92), the autoencoder (0.83) and

AE1SVM (0.78) do as well, and LUNAR (0.63) follows; Deep SVDD (0.13) does not. KNN (0.39) and PCA (0.48) are below the forest. ECOD and COPOD, which look at each column alone, still reach 0.52 and 0.51, which I had not expected; probably the injected combination also puts one of the values in a tail of its own column. HBOS, built on histograms of single columns, stays at 0. The precision of Isolation Forest and the One-Class SVM on this type is only about 0.22: they cast a wide net that also catches clean rows.

- *Invalid formats* are caught mainly through the impossible numeric value they carry (IQR recall 1.00, precision 0.37; Z-score precision 1.00; DIF 1.00, LUNAR 0.90, KNN 0.81); the malformed string fields are missed by all.
- *Temporal drift* is the weakest column for most methods. LUNAR is the best (0.65), ahead of LOF (0.50), the best of the first six; every other method stays below 0.42. The dedicated *drift* detector scores 0 here too: at the reference magnitude the shifted values stay inside the global range, and once diluted by the many clean recent rows the window mean has moved by less than a tenth of a standard deviation on average (0.04 to 0.11 over the five test splits). The present row-level drift detector remains at zero even in the severity sweep; the windowed-monitoring extension is separate.

The practical reading is a short decision list: null-rate problems to declarative rules, single-column outliers to any statistical rule, implausible combinations to a tree-, boundary- or reconstruction-based model (Isolation Forest, the One-Class SVM, DIF, the VAE), and duplicates and drift to dedicated stages the detectors do not replace, with LUNAR as the one learned model that helps on drift at the price of more false alarms (Section IV-F).

D. Detection rate versus injection severity (RQ2)

The synthetic magnitudes are a design choice, so I sweep them. A single *severity* multiplier scales the univariate factor, the multivariate percentile extremity and the temporal-drift multiplier (1.0 is the dataset used above). Fig. 2 plots per-type recall against severity. Univariate outliers are already saturated at severity 0.5. Multivariate outliers show a threshold: Isolation Forest and the One-Class SVM climb to recall 1.0, while IQR jumps from 0.04 to 1.0 once the injected percentile crosses its fence. Temporal drift becomes easier for IQR, Isolation Forest and One-Class SVM (roughly 0.2–0.3 recall at severity 1.0 and 0.6–0.7 at severity 3), while LOF peaks earlier and then falls. The row-level drift detector does not respond in this sweep. The ranking therefore changes with severity, limiting claims based on one injection magnitude.

E. Further schemas and neural models (RQ2)

a) Two further generators.: A table of card transactions and a table of sensor readings keep the structural rows (Great Expectations finds every missing value and the deduplication rule every duplicate, on all three schemas) and the overall profile: the rank correlation between the method-by-type matrices

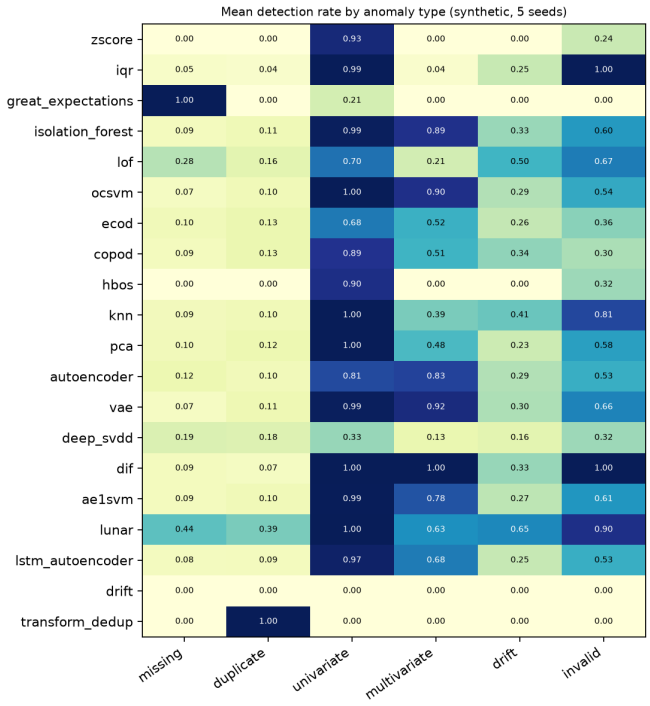


Fig. 1. Mean detection rate (recall) over five seeds of each method on each injected anomaly type, synthetic test split. Darker is higher. `transform_dedup` is the deduplication rule of the transform stage, shown for reference.

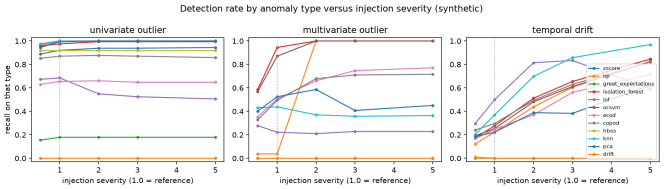


Fig. 2. Per-type recall against injection severity (1.0 is the reference synthetic dataset), mean over three seeds. The dashed line marks severity 1.0.

of two schemas is 0.93 on average for the first six detectors and 0.89 with all seventeen. They do not keep the values. For multivariate outliers, Isolation Forest and the One-Class SVM reach 0.89 and 0.90 on the customers but 0.49 and 0.18 on the transactions and 0.32 and 0.33 on the sensors; for drift, the best recall is 0.50, 0.38 and 0.96, the sensors being the case where a factor of 1.5 on a signal of about -70 dBm moves values several standard deviations away. Two of the four hypotheses hold (structural rows, rank stability). The injected magnitudes are relative, so how visible drift and multivariate outliers are depends on how a column is scaled and correlated, not only on the type (Table XVI of Appendix D).

b) Neural models and further classical detectors.: On the customer matrix the autoencoder detects multivariate outliers (0.83 ± 0.10) about as well as Isolation Forest (0.89), and an LSTM autoencoder trained to convergence does not catch drift better than the row-level detectors (0.25 ± 0.07 against 0.50 for LOF); three of the four hypotheses of that study hold.

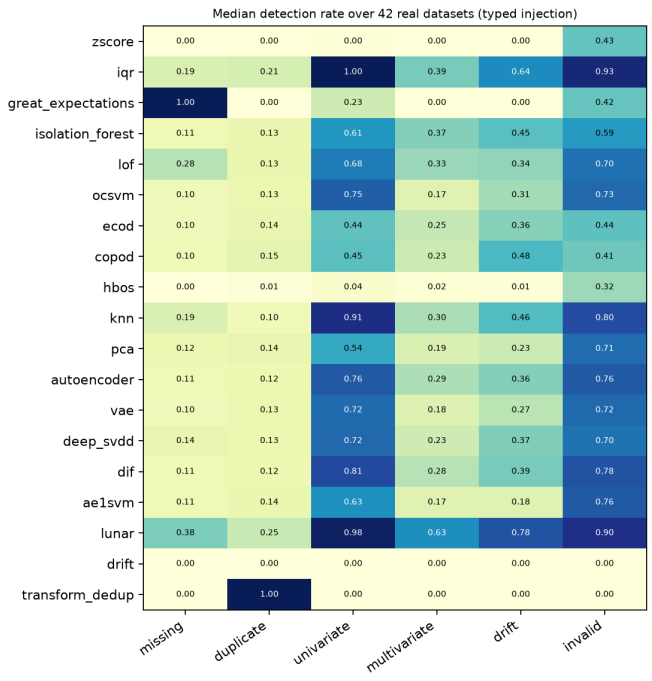


Fig. 3. Median detection rate over 42 real datasets with typed injection. Compare with Figure 1.

The complete registry then adds five classical and five neural detectors, and none of its three hypotheses on this matrix is supported: ECOD and COPOD reach 0.52 and 0.51 on multivariate outliers (I expected at most 0.20), PCA reaches 0.48 against 0.83 for the autoencoder, and LUNAR beats LOF on drift (0.65 against 0.50) but flags 41% of the clean rows, against 9% to 16% for the other neural models. DIF reaches 1.00 on univariate and multivariate outliers and on invalid formats with 9% of the clean rows flagged; the VAE (0.92) and AE1SVM (0.78) follow the autoencoder on multivariate outliers, and Deep SVDD is the weakest neural model. The detailed text is in Appendix D.

F. Typed injection on real tables (RQ2)

The synthetic matrix of Section IV-C comes from a single generator. Section III-H describes how the six anomaly types were injected into 42 real UCI tables; this subsection reports what the detectors find there.

Two rows of the synthetic pattern hold on every table: Great Expectations finds all missing values and the deduplication rule all duplicates. The rest does not carry over. Three of the five hypotheses are not supported (Great Expectations and deduplication aside, the specialisation, the multivariate gap and the difficulty of drift). On multivariate outliers the median detection rate is 0.37 for Isolation Forest and 0.17 for the one-class SVM, against 0.89 and 0.90 on the synthetic data. The interquartile fences reach the best detection rate on several types, but they also flag a median of 22% of the clean rows (up to 99% on some wide tables), so the detection rate alone favours them. The z-score detects almost nothing on univariate outliers (median 0.00 against 0.93 synthetically). A check made after seeing this, and therefore exploratory, shows why: the invalid rows of the training split inflate the standard deviation, and without them the z-score detects every univariate outlier on all 42 tables. The result says more about non-robust statistics on a contaminated training set than about the z-score itself.

The two hypotheses on stability hold. The mean rank correlation between the method-by-type matrices of two tables is 0.80 with the seventeen detectors of the registry (it was 0.83 with the first eight rows), and it is lowest for drift (0.63), multivariate outliers (0.65) and duplicates (0.66), highest for univariate outliers (0.76). Part of this stability is mechanical, since many cells are close to 0 or 1. The detection rate of Isolation Forest on univariate outliers falls with the number of continuous columns (rank correlation -0.96 over the 42 tables). The anomalies are injected, not natural, and at one severity.

The classical and neural detectors added to the registry were run on the same 42 tables (Figure 3), without a hypothesis of their own on this study. On multivariate outliers LUNAR has the highest median detection rate (0.63, Isolation Forest 0.37, KNN 0.30, the autoencoder 0.29, DIF 0.28, ECOD 0.25), and also on drift (0.78 against 0.64 for the IQR rule and 0.45 for Isolation Forest) and on missing values among the learned models (0.38, Great Expectations 1.00). It pays for it: it flags a median of 24% of the clean rows (97% on the worst table), where the other learned models, calibrated to the same training prevalence, flag 12% to 15% and HBOS 1%. Only the IQR rule and LUNAR flag more than a fifth of the clean rows on median. The column-wise ECOD and COPOD detect univariate outliers less often than KNN (0.44 and 0.45 against 0.91) and HBOS almost never (0.04). So the synthetic reading holds in outline: no detector covers every type, and the one that comes closest buys its recall with false alarms.

G. Precision-recall curves and the operating point (RQ3)

Fig. 4 draws the full precision-recall curve of every detector on each dataset's test split. The curves make the aggregate tables concrete: on Cardio the Isolation Forest and One-Class SVM curves stay high (AP 0.80 and 0.76); on Water Quality every curve hugs the 0.61 base-rate line; on Credit Card Fraud Great Expectations and Isolation Forest dominate but neither clears a precision of 0.4 except at negligible recall. Table IV makes the point numerically for Isolation Forest: as the contamination moves from 0.001 to 0.20, its Credit Card Fraud precision falls from 0.37 to 0.008 while recall rises from 0.23 to 0.92; on Cardio the same sweep trades a precision of 1.00 at recall 0.04 for a recall of 0.93 at precision 0.46. The estimator is fixed; only the threshold moves. Reading the prevalence off the training split lands near the knee of the curve on the datasets where a knee exists.

H. Fixed contamination (RQ3)

The train-prevalence operating point uses labels from the training partition. To measure its influence, I reran the three

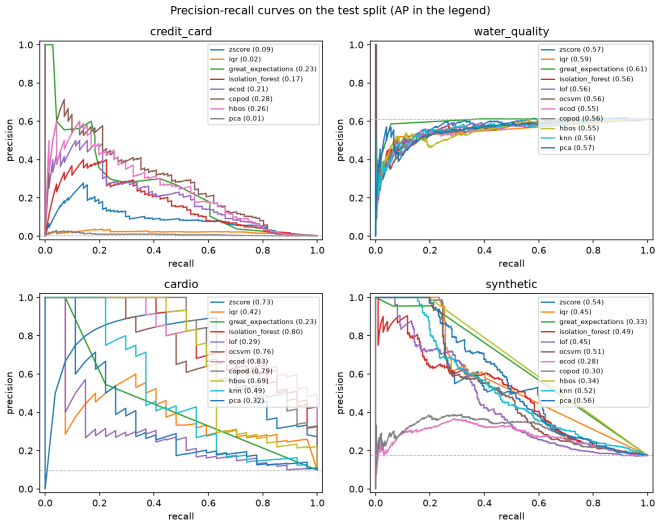


Fig. 4. Precision-recall curves on the test split, one panel per dataset, average precision in the legend. The dashed line is the base rate.

TABLE IV
ISOLATION FOREST UNDER A CONTAMINATION SWEEP, TEST SPLIT, ONE SEED. "PREV." IS THE TRAINING-SPLIT PREVALENCE, THE VALUE THE BENCHMARKS USE.

Dataset	Contam.	Prec.	Rec.	Flag.%
Credit Card Fraud	0.001	0.37	0.23	0.10
Credit Card Fraud	0.10	0.01	0.90	10.2
Credit Card Fraud	prev. 0.002	0.28	0.32	0.20
Cardio	0.01	1.00	0.33	3.3
Cardio	0.20	0.46	0.93	19.7
Cardio	prev. 0.096	0.72	0.67	9.1
Water Quality	prev. 0.5	0.58	0.49	51.6
Synthetic	prev. 0.175	0.53	0.51	16.8

example datasets with the same five seeds and partitions, replacing that value by a constant contamination of 0.02 for the ML detectors. Table V shows Isolation Forest, the only learned detector run on all three datasets. The ranking score and its average precision are unchanged; only the binary decision threshold moves. On Credit Card Fraud, the fixed setting raises recall from 0.206 to 0.682 but lowers precision from 0.202 to 0.055. On Cardio, it raises precision from 0.639 to 0.983 but lowers recall from 0.615 to 0.267. The Water Quality target is a majority class, so a 2% outlier budget leaves almost all positive rows unflagged. The fixed value was chosen before these comparator runs but after the baseline results were available. These numbers are therefore a post-hoc sensitivity analysis, not a new confirmatory estimate on an untouched test set.

I. Operating point in time (RQ3)

a) Chronological pilot on Credit Card.: Fitting on the oldest 70% of the rows and scoring the newest 15% lowers the average precision of the first four detectors, but the interval excludes the stratified mean only for Great Expectations and

TABLE V
ISOLATION FOREST WITH TRAIN-PREVALENCE AND FIXED 0.02 CONTAMINATION, SAME FIVE REAL-DATA SPLITS. ENTRIES ARE MEANS; "FLAG" IS THE FRACTION OF TEST ROWS PREDICTED ANOMALOUS.

Dataset	F_1 train	F_1 fixed	Flag train%	Flag fixed%
Credit Card Fraud	0.203	0.102	0.168	2.063
Cardio	0.618	0.406	9.854	2.701
Water Quality	0.512	0.023	49.675	1.789

Isolation Forest, and part of the gap may come from a change of prevalence (0.18% in training, 0.12% in the test period, 52 frauds). Isolation Forest keeps a high ROC-AUC (0.936) but flags 0.08% of the future rows and none of the frauds: the threshold set from the training scores does not carry over. HBOS is the one column-wise detector whose training-prevalence threshold still catches frauds in the later period (average precision 0.151, precision 0.26, recall 0.19), and PCA collapses (0.002); the intervals are wide, so this shows that the column-wise detectors are not alike under a time split, not that HBOS is better.

b) Decisions under temporal shift.: On Bank Marketing the prevalence goes from 5.8% in training to 11.5% in validation and 39.3% in the test period. The average precision under the chronological split is higher than under the stratified one for every detector, but the ROC-AUC is between 0.48 and 0.63; matching the control to the test prevalence cuts the median gap from 0.26 to 0.06, so the apparent gain is a change in prevalence, not in detection. On Credit Card the same matching reduces the gap from 0.053 to 0.031, less than half of it. A label-free rolling threshold that keeps the training prevalence cannot follow a prevalence that changes: on Bank Marketing it flags 1.7% and 6.3% of the rows and drops the F_1 to 0.02 and 0.11, where the training threshold had 0.51 and 0.41 by accident of a prevalence of 39%. On Credit Card no detector finds a fraud with the training threshold, and the rolling one gives a recall of 0.12 to 0.14 at a precision below 0.08. One hypothesis of four holds: a threshold fixed on the past can fail in both directions, and part of what a chronological split changes is only the prevalence.

c) Sequence model.: An LSTM autoencoder reading the ten previous rows of the stream reaches an average precision of 0.037 ± 0.006 on the 52 test frauds against 0.024 ± 0.010 for the row-level autoencoder. This is comparable to Isolation Forest (0.035) and Great Expectations (0.045), not clearly better, and none of the five trainings converged within the budget. The protocols and the detailed tables are in Appendix D.

J. Cost in the pipeline (RQ4)

A growth experiment on 100 to 900 rows cannot be extrapolated, so I measured the detectors again on synthetic matrices of up to 300,000 rows (protocol in Section III-H). Times are those of the detection stage (fit, predict and score).

The rule-based detectors and Isolation Forest grow roughly linearly (exponents below one at small sizes reflect fixed start-up costs), LOF grows with an exponent near 1.5 and finished

TABLE VI

MEDIAN DETECTION-STAGE TIME ON SYNTHETIC DATA (8 FEATURES) FOR 10^4, 10^5 AND $3 \cdot 10^5$ ROWS, THE EMPIRICAL GROWTH EXPONENT OVER THE SIZES COMPLETED, AND THE PEAK RESIDENT MEMORY ADDED BY DETECTION AT THE LARGEST SIZE (A: AT $3 \cdot 10^4$ ROWS, B: AT 10^5 ROWS, THE LARGEST SIZE COMPLETED).

Detector	10^4	10^5	$3 \cdot 10^5$	Exp.	Mem. (MB)
Z-score	0.002 s	0.013 s	0.039 s	0.64	70
IQR	0.004 s	0.029 s	0.086 s	0.70	116
Isolation Forest	0.16 s	1.09 s	3.2 s	0.72	140
Autoencoder (MLP)	1.5 s	8.4 s	12.7 s	0.77	115
LSTM autoencoder	1.1 s	5.1 s	7.3 s	0.73	499
LOF	0.73 s	18.0 s	133 s	1.51	405
One-Class SVM	5.6 s	$>$ 300 s	not run	1.99	788[a]
ECOD	0.045 s	0.68 s	2.35 s	1.10	186
COPOD	0.044 s	0.68 s	2.36 s	1.09	186
HBOS	0.013 s	0.12 s	0.39 s	0.87	94
PCA	0.010 s	0.089 s	0.29 s	0.85	97
KNN	0.45 s	11 s	68 s	1.46	148
VAE	6.83 s	69 s	209 s	1.00	105
Deep SVDD	5.65 s	57 s	192 s	1.01	383
DIF	23 s	262 s	$>$ 300 s	1.01	2748[b]
AE1SVM	22 s	213 s	$>$ 300 s	0.99	62[b]
LUNAR	29 s	$>$ 300 s	not run	0.99	369[a]

$3 \cdot 10^5$ rows in about 133 seconds, and One-Class SVM, with an exponent near two, exceeded the budget at 10^5 rows and was not tried beyond. The LSTM autoencoder was trained for at most five epochs on at most 50,000 rows, a budget cap: only its scoring cost grows with the input after that. These exponents describe the sizes measured and are not extended beyond them. No measurement was made under contention as defined in the protocol, but the machine was not isolated.

The ten detectors added later (`verification/full_registry.md`) were timed on the quiet machine with the same protocol; seven of the 252 runs were flagged as contended and are kept in the archive. Here the neural models are fitted on all the rows, without the 20,000-row cap of the quality studies, so their time grows with the input. ECOD, COPOD, HBOS and PCA take a few seconds at $3 \cdot 10^5$ rows, KNN about a minute (exponent 1.46, like LOF), and the VAE and Deep SVDD three to four minutes. DIF, AE1SVM and LUNAR exceeded the budget of 300 seconds at $3 \cdot 10^5$, $3 \cdot 10^5$ and 10^5 rows. The cost of a neural model is therefore two orders of magnitude above the column-wise models, and the cap used for quality is what made the pool runs feasible.

To separate the detection stage from the whole pipeline, I also timed read, cleaning, detection and write on the full Credit Card file (284,807 rows read, 283,726 after removing duplicates). With the Z-score the pipeline takes about 4.8 seconds, of which detection is 0.13 seconds (3%) and writing the two sorted files is 3.9 seconds; with Isolation Forest it takes about 8.5 seconds, of which detection is 3.7 seconds (43 to 45%). Peak memory is about 0.75 GB and 0.95 GB respectively. Detection-stage latency is therefore not the latency of the whole pipeline, and the gap depends on the detector. These are single-machine figures on synthetic data and one real file, not a general ranking.

TABLE VII

THE SIX HYPOTHESES OF THE WHOLE-POOL EXTENSION, WRITTEN BEFORE THE RUNS, AND THE VALUE THAT DECIDES EACH ONE.

Hypothesis	Outcome	Value
P1 label-free threshold lowers the median F_1 (at least 12 of 17)	holds	14 of 17
P2 ranking by curves equals the AP ranking ($\rho \geq 0.99$)	not supported	$\rho = 0.90$
P3 recall rises with severity for every detector	not supported	13 of 17
P4 detection above 30% (Isolation Forest) and below 10% (Z-score) of the pipeline on large datasets	holds	62% and 4%
P5 chronological AP lower for two thirds of the pairs	not supported	31 of 64
P6 no pool file differs from the manifest	holds	152 of 152

K. The studies extended to the whole pool

The curves, the contamination sweep, the label-free threshold, the severity sweep, the pipeline run and the chronological split were first done on a few datasets. I applied each of them to every pool dataset for which it is defined (Section III-H). Table VII gives the outcome of the six hypotheses written before the runs; the tables that support them are in Appendix D, and the datasets for which a study is not defined are listed with their reason in the archive.

a) Eligibility.: The eligibility rule of Section IV-F applied to all 152 datasets, ADBench ones included, leaves 58 eligible tables, 42 from the UCI repository and 16 from ADBench; the others have fewer than four continuous columns (categorical or binary tables) or too few rows. Pooling the 58 tables changes nothing of the picture of Section IV-F: the mean rank correlation between the matrices of two tables is 0.80 (0.80 on the UCI tables), the rank correlation between the number of continuous columns and the univariate recall of Isolation Forest is -0.96, and the same three of the five hypotheses are not supported.

b) Label-free threshold and curves (P1, P2).: A constant contamination of 0.02, read from the stored training scores, lowers the median F_1 of 14 learned detectors; the three rules do not read the contamination and are unchanged. The ranking by the average precision of the curves (seed 42) correlates with the published AP ranking at 0.90: several detectors have medians within a hundredth of each other and swap places between one seed and five. On Isolation Forest the median F_1 keeps rising up to the largest contamination of the grid, 0.2, twice the median prevalence of the pool (0.10), while the precision stays between 0.14 and 0.16 from 0.05 on: the scores are weak on these labels, more than the threshold is badly placed (Tables XVII and XVIII).

c) Severity (P3).: Over the 58 eligible tables, 13 of 17 detectors gain recall on multivariate outliers between the smallest and the largest amplitude, and Z-score, Great Expectations, LOF, HBOS do not. The Z-score, the Great Expectations bounds and HBOS stay at zero because they read

one column at a time, and multivariate outliers stay inside the range of each column, as in the per-type matrices; LOF is already at its level at the smallest severity. LUNAR is the highest at every severity (Table XIX).

d) The pipeline end to end (P4).: The pipeline ran on all 152 datasets with Z-score and Isolation Forest; 14 of the 304 runs failed and are recorded with their reason (a table with no numeric column, or a column that the pipeline reads as text because of stray header lines in the file). On the 14 datasets above 50,000 rows the detection takes 62% of the pipeline time with Isolation Forest and 4% with the Z-score, the writing of the two files being the rest, as on Credit Card Fraud. These times were measured while other jobs ran on the machine, so they are orders of magnitude, and the shares are ratios inside one process.

e) Chronological split (P5).: Only some pool datasets have a time variable: eleven unlabelled UCI tables have a date, the ADBench files have none, and five labelled datasets qualify (Credit Card, Bank Marketing, Occupancy Detection, Online Shoppers and Room Occupancy). The AP under the chronological split is lower than under the stratified one for 31 of 64 pairs of a dataset and a detector, not for two thirds. Bank Marketing is the reason: its prevalence rises from 5.8% to 39.3% and no detector loses AP there, as Section IV-I found with four detectors. Room Occupancy is excluded because its anomalies are all in the oldest rows (Table XX).

f) Files (P6).: All 152 local files match the pinned manifest (`make verify-pool`).

V. Discussion

a) Answers to the research questions.: RQ1. In one pipeline the families are close on aggregate scores. On the 100 labelled problems no detector is clearly ahead among the seventeen, the column-wise ECOD, COPOD and HBOS match Isolation Forest, and the neural models do not beat the best classical one by the margin I had set (0.02 of median AP). *RQ2.* Per type, the competence of each family is narrow: declarative rules own missing values, statistical rules own single-column outliers, models that read several columns own implausible combinations. The profile holds in structure across three generators and on 42 real tables, but the values of the multivariate and drift cells depend on how the columns are scaled and do not transfer. *RQ3.* The operating point moves precision and recall by an order of magnitude with the same scores, a label-free contamination lowers the median F_1 of fourteen of the seventeen detectors, and a threshold fixed on the past fails in both directions when the prevalence moves; only 31 of 64 pairs of a pool dataset and a detector lose AP under a chronological split, so that effect is not general. *RQ4.* Detection is cheap for the rules and the column-wise models, one to two orders of magnitude more for the neural ones, and it is not the cost of the pipeline: writing the output dominates for the rules, while detection takes 62% of the time with Isolation Forest on the datasets above 50,000 rows.

b) Rules and learned models are complementary, not competing.: The per-type heatmap shows limited overlap between what the declarative rules catch (missing values), what the statistical rules catch (single-column outliers), and what the learned models add (implausible combinations, recall 0.89 for Isolation Forest against ≤ 0.04 for the rules, and 1.00 for DIF and 0.92 for the VAE among the neural models). Broad coverage needs more than one family, not the single best one on aggregate. This is a different conclusion from benchmarks that publish a single ranking.

c) The operating point matters as much as the algorithm.: Setting Isolation Forest's contamination to the data prevalence rather than a fixed 0.10 moves its fraud precision from 0.01 to 0.28 (the seed of Table IV; 0.20 on average over five seeds) and its multivariate-outlier recall on the synthetic data from about 0.3 to 0.9 (means over five seeds). The same estimator looks weak or strong depending on one number that a benchmark must choose deliberately.

d) Unsupervised detection works in one of the three regimes.: The three example datasets bracket the method: on Cardio, where anomalies are rare and distributionally distinct, Isolation Forest reaches $F_1 = 0.62$ and AP 0.70; on Credit Card Fraud, where the positive class is a 0.17% mixed-defect label, the best F_1 of the seventeen detectors is 0.35 (Deep SVDD, precision 0.26) and HBOS reaches the best precision (0.34) at a recall of 0.32; on Water Quality, where the "anomaly" is a 61% majority, every method is at chance. The question is less "rules or ML" than "does this dataset have rare, outlying anomalies at all".

e) The profile transfers in part, and the decision does not.: The later studies test whether the synthetic profile is an artefact of one generator. Two further schemas keep the ordering of the methods (rank correlation 0.93), so the competence of each family is not specific to the customer table; but the multivariate and drift cells depend on how the columns are scaled, so their values do not transfer. On the 42 real tables the structural rules still own their types while the multivariate gap shrinks, and on the 100 labelled problems the medians of AP of One-Class SVM (0.240), Isolation Forest (0.215) and the Z-score (0.203) overlap, so the advice "use a learned model" is not supported there. The neural models add little on aggregate: the best of them (DIF, 0.226) stays below the best classical median (0.240), and the LSTM autoencoder recalls 0.25 of the drifted rows against 0.50 for LOF; LUNAR is the exception on drift (0.65 on the customers), at the price of flagging 41% of the clean rows. Last, a ranking that holds under a chronological split does not give a usable threshold: on Bank Marketing the prevalence goes from 5.8% to 39.3%, matching the control to it removes most of the apparent gain (0.26 to 0.06), and a label-free rolling threshold cannot follow it. A practitioner should therefore choose the family from the type of fault and monitor the prevalence before trusting any alert rate.

f) A larger registry does not produce a winner.: The registry now holds three rules, eight classical models and six neural ones. On the 100 labelled problems every one of the seventeen is first on at least two, and no method is first on more than 13. The column-wise detectors ECOD,

COPOD and HBOS cost little, match Isolation Forest on the pool (0.240, 0.216 and 0.237 against 0.215) and beat it on Credit Card, but they cannot see an implausible combination of ordinary values; on the synthetic data they only do because the injected combinations also push one value to a tail. The neural models help on specific types (DIF and the VAE on multivariate outliers, LUNAR on drift) and not on aggregate scores, and they cost more to fit and to tune. For a practitioner this suggests a cheap column-wise model and Isolation Forest as the baseline, a reconstruction model added when the faults are combinations, and a model such as LUNAR only when missing drift costs more than false alarms. These are default settings, and a tuned neural model could do better.

g) Aggregate metrics on a mixed label are misleading.: On every dataset the aggregate table compresses different behaviour into similar F_1 and AP values. A method excellent on one anomaly type and blind to the rest scores like a mediocre all-rounder. The controlled per-type dataset, and the severity sweep that tests the sensitivity of that ranking, are what make the difference measurable.

h) Pipeline integration changes the shortlist.: Latency and scaling matter, but the timing in these runs is too variable to establish a reliable speed ranking. The One-Class SVM and LOF were omitted from the full fraud dataset because their computational costs need a separate scaling study. I time only the detection stage (fit, predict and score) inside the pipeline; extraction, transformation and loading are outside the measurement, so these figures are not the latency of a whole ETL run.

i) Threats to validity of the results.: The synthetic dataset still encodes my own assumptions about what each anomaly type looks like; the severity sweep addresses the magnitude choice but not the shape of the injections. The real-data pool answers part of this, but its anomalies are either injected (the 42 tables) or taken from a rare class (the 100 labelled problems), so none of them is a data-quality fault observed in a production pipeline. The ML detectors carry a wide seed spread on the small datasets (Cardio F_1 standard deviation up to 0.17; Isolation Forest on fraud $F_1 = 0.20 \pm 0.07$), so those numbers should be read as ranges. The labels of the 64 UCI problems come from a rule (the rarest class), 19 sources appear in both lists of the pool, and the datasets are therefore not independent. The labelled-pool preparation also reads full-table values and labels before the split; the exploratory training-only path has only constructed validation so far. Its performance impact needs a separate replay, not a retrospective reinterpretation of the archived outcomes.

j) Limits of the verification.: Complete line and branch coverage does not establish that the scientific assertions detect every defect, and mutation testing cannot show whether the injected defects and the tests cover every way an experiment can go wrong. The checks were run on one machine by the person who wrote the code, and the timings do not transfer to another. The details, the history of the replays and the gaps of the real-data check are in Appendix B; they leave some implementation errors capable of changing a published result without being caught by the current gates.

VI. Conclusion

I built an ETL pipeline in which three rule-based detectors, eight classical machine-learning detectors and six neural ones share one interface and are scored on the same held-out path, over five seeds, with the operating point read from the data. A controlled dataset labels six anomaly types separately, and a severity sweep shows where the per-type ranking changes as the injections are scaled. On aggregate metrics the detector families stay close on the example and synthetic datasets, and on 100 labelled problems of a pool of 152 open tables the historical rankings require train-only revalidation (Section IV-B). The per-type view is where they diverge: declarative rules own missing values, statistical rules own single-column outliers, tree-, boundary- and reconstruction-based models (Isolation Forest, the One-Class SVM, DIF, the VAE) catch implausible combinations, and duplicates and temporal drift need dedicated stages, LUNAR being the one learned model that helps on drift, at the price of many false alarms. Across the three example datasets, unsupervised detection pays off only on Cardio, where the anomalies are genuinely rare and outlying. The choice of contamination, the precision-recall trade-off it selects, and the pipeline cost further constrain which detector fits a real flow.

On real tables the pattern is less clean than on the generator. With the six types injected into 42 tables, Great Expectations and the deduplication rule still own their types, but the multivariate gap shrinks and the z-score collapses for a reason that has to do with its training data, not its design. On the 100 labelled problems, the descriptive results are conditional on historical pre-split preparation; they do not establish a validated train-only ranking or equivalence among the seventeen detectors. Two further schemas keep the profile (rank correlation 0.93) but change how visible multivariate outliers and drift are, and an LSTM autoencoder trained to convergence does not catch temporal drift better than the row-level detectors. The chronological pilot and the study of decisions under temporal shift, on Credit Card and Bank Marketing, point the same way: score rankings survive a split by time better than a threshold fixed on the past. On Bank Marketing, where the prevalence goes from 5.8% to 39.3%, matching the control to it cuts the apparent gain of the chronological split from 0.26 to 0.06 of AP, and a label-free rolling threshold that keeps the training prevalence cannot follow it; only one of the four hypotheses of that study holds.

The next steps are a check of the injected faults against faults observed in real pipelines, and a decision-level study on more datasets with genuine time, such as the Kyoto-2006+ traffic, whose data rights I have not verified. The open questions are listed in Section V.

Appendix A
Verification design

The experimental code is checked at three levels. A routine test suite exercises numerical contracts on small, generated

tables and measures line and branch coverage across the pipeline and experiment scripts. Its regression cases include partition disjointness by customer identity, training-only preprocessing, the exclusion of labels from detector inputs, score direction and index alignment, undefined binary metrics, and the construction of each synthetic anomaly type. Separate command-line tests run the four figure and table generators on temporary data. They check the output schema without comparing images pixel by pixel or imposing a fixed wall-clock duration.

A second, manually invoked level asks whether these checks can fail. Mutation testing generates small source-level changes (a reversed comparison, an altered constant, a modified message) and counts how many make the routine suite fail. I apply it to the modules that compute a result and to the experiment scripts. In addition, twenty-three deliberate scientific defects are applied one at a time to a temporary copy of the code, and a named test must fail on each: a scaler refitted on the rows being scored, the label passed to the detector as a feature, a contamination threshold read from the test labels, a training row copied into the test partition, an inverted score orientation, ranking metrics computed from the binary flags, a seed aggregate that keeps only the first seed, a detector missing from the results table, and a few others.

A third level uses the locally available full datasets. A separate command compares, for each of the three research CSV files, the file checksum, the raw and post-deduplication row counts, the number of anomalies and the positive-class value with an archived manifest. An absent file is reported as not run, and the command as a whole passes only if all three files pass. The scripts record the dataset fingerprint, software versions, random seeds, split policy and output names alongside their results. For each seed on the three main real-data datasets, an archive stores the positions of the train, validation and test rows after deduplication; a capped fit also records the selected training positions. The historical pool runs retain seeds but do not export equivalent source-bound row-position manifests; recovery remains outstanding. Archived tables and figures are checked against their manifest, and the six numerical tables in this manuscript are compared with the archived CSV values at the precision shown. This establishes traceability between the reported values and the recorded experiment; it does not establish that another machine will reproduce the same timings.

A fourth part turns these checks into a release verdict. Each profile (routine suite, mutation campaign, fault injection, real-data check, coverage of the checking scripts themselves, static typing, and a controlled reproduction) writes a small evidence record tied to the source revision, to a clean working tree, to a fingerprint of the code that computes results (the pipeline and the experiment scripts, not the checking scripts) and to the software versions. A release script recomputes every status from the raw results instead of reading it from the record, requires that all profiles describe the same revision and compatible environments, and returns *certified*, *incomplete* or *rejected*; figures quoted in this manuscript are accepted only if the run they come from is not older than a change to the code. The real-data check can also run on a hosted Linux machine: it downloads the three public files, compares each SHA-256 value with the archived manifest before using it, and refuses any other file. The controlled reproduction replays the cardiotocography benchmark against the archive, twice in the same environment and once in an environment rebuilt from a recorded, hash-pinned lock; on another operating system the metrics agree statistically (largest difference 6.5×10^{-5}, identical partitions) rather than bit for bit. Result files are written to a temporary name and renamed only when complete, so an interrupted run cannot leave a truncated table or figure. The verdict states where the evidence was produced and never that it was independently attested: the author wrote the code and launched every run.

I treat a failed or missing check as a limitation of the evidence, not as a successful experiment. Complete line and branch coverage and a mutation score of at least 95% over the pipeline and the scripts are the targets, and the measured values, with what they leave out, are reported in Appendix B. The chronological split and the window-based drift monitor have a separate protocol: their alert and delay measures are not pooled with the row-level metrics from the stratified benchmark.

a) Prospective risk-tiered policy.: A prospective verification revision assigns minimum mutation scores of 80% (L1, limited consequences), 90% (L2, operational support), 95% (L3, scientific processing), and 98% (L4, leakage, metric/evidence integrity and certification). Each required module must meet its own threshold; aggregates cannot compensate for a failing module or risk level. Classification uses the highest applicable consequence. The explicit four-root inventory and the per-module campaign gate are implemented prospectively; complete release integration and independent review are not yet established. The historical results below retain their original global 95% criterion and are not claims of compliance with this revision. A first complete native campaign over the governed scope ($29{,}811$ mutants, one clean environment) killed $19{,}248$; the L4 aggregate was 65.5% against the required 98%, and the gate failed. A further $7{,}501$ mutants had no covering test. This is a measured diagnostic of the current test suite, not a certification. The native-catalog adapter distinguishes assertion kills from internal runner errors and checks generated identities against source-function hashes. A separate execution-binding prototype tracks source globals, tests, configuration and runtime metadata. Unchanged bound verdicts can be reused; input changes following an interrupted attempt require conservative invalidation when dependency maps may be partial. Neither catalog parsing nor these constructed acceptance checks establishes current release compliance. Changes to compliance thresholds alone require a new policy evaluation rather than repeating unchanged execution. A constructed native smoke check executes two mutants and subsequently reuses both unchanged verdicts. Per-attempt logs are preserved, while live workers, changed inputs or stale generation artifacts prevent a new execution binding.

This small check does not measure mutation adequacy of the benchmark modules. Recorded arithmetic can be recomputed without the historical source files; current scope and execution bindings must still be verified before reuse.

APPENDIX B
VERIFICATION OUTCOMES

The exploratory pool-preparation modules have a separate constructed acceptance suite and an explicit line/branch coverage gate. Three deliberately introduced schema/scaling/label-access defects are detected. This is bounded validation of the new path, not replay evidence for the historical pool. The routine and mutation figures below describe the named archived run and do not extend that mutation denominator to these new research modules.

At commit d4304c5b (local run, not independently attested) the routine suite runs 1,292 tests, plus three that need the full datasets and twenty-three fault-injection runs, all run separately. Coverage of the pipeline modules and the experiment scripts is complete: 1,790 of 1,790 executable lines and 350 of 350 branches. The tools that check the checks (the evidence and release scripts under `verification/`) are measured apart, at 2,026 of 2,026 lines and 738 of 738 branches, and are not part of that figure. The mutation campaign on the pipeline modules and the experiment scripts kills 4,980 of 5,102 mutants (97.61%). Split by what the code does, the modules that compute a result (splitting, metrics, the synthetic generator, the detectors) kill 1,927 of 2,014 (95.68%) and the input, output and script code kills 3,053 of 3,088 (98.87%), so the aggregate is higher than the score of the scientific computation alone. The 122 survivors count as failures. I classified them by reading the changes, not by proof: most pass an explicit default (`dtype`, `index`) or change an internal label. Two that I first took for unreachable, a cap on a simulated transaction rate, turned out to be reachable by a very rich customer and are now killed by a test.

All twenty-three deliberate defects are caught. The first fourteen were not always caught by the tests I started with: four went through every existing test (a scaler refitted on the scored batch, ranking metrics fed with binary flags, a detector missing from the results, and a contamination threshold read from the synthetic test partition), and a fifth, an inverted Z-score, was caught only by a test added during the mutation work. None of these defects was in the code; the tests were missing. After adding them I reran every experiment: all detection metrics are unchanged and only the timings differ. The nine defects added later (a chronological split that trains on the latest events, validation rows copied into training, an undefined precision reported as one, inverted machine-learning scores, scores reordered before the comparison with the labels, a contamination read from train and test labels, a silently dropped seed, a real-data check that passes without checking the files, and an interrupted mutation campaign accepted on its score) were each caught by a named test. The real-data checks pass locally on the three CSV files, and the six tables of this manuscript match the archived results.

The same profiles were run on a hosted Linux machine for the final revision (an earlier revision, commit c64c6ffa, a continuous-integration run, not independently attested). The release script returned *certified*. The routine suite and the scripts keep complete line and branch coverage, the checking scripts are fully covered (1,975 of 1,975 lines), and the mutation campaign, run from scratch, kills 4,703 of 4,808 mutants (97.82%) with no timeout. Earlier hosted runs reported fourteen timeouts, always the same mutants. They were not hangs in the mutated code: the mutation tool starts worker processes after the numerical libraries have created their thread pools, which blocks on Linux. Running the tool with single-threaded libraries removed them, and the same four mutants then gave the same outcomes as on macOS. The three real datasets were downloaded and matched the archived checksums, and the controlled reproduction passed at its three levels. These results show that the evidence is consistent and re-creatable on a second platform; they do not certify the scientific conclusions of the study, which rest on the limits stated with each experiment.

a) Replay history and gaps.: The previous archive was made on a working tree with uncommitted changes (revision `841c3aa`). The archived runs were therefore replayed from a clean commit (`93b6fe8`): every detection metric, curve and partition is identical to the previous archive and only the timings changed, which is why the two timing tables and the scaling figures were updated. The new records name that commit and mark the working tree as clean, but they were produced on one machine. The complete real-data checks were performed with the three files available locally. The Water Quality file matches its named mirror, and the Cardio CSV can be reproduced from the current ADBench archive. The Credit Card file has the size of the current Kaggle version (3, March 2018), but I could not compare its content hash with a fresh download. These gaps leave some implementation errors capable of changing a published result without being caught by the current gates.

Detector timing in Tables XII and XV sums `fit`, `predict` and `score` inside the detection stage; CSV loading, deduplication and figure rendering are excluded. A separate local growth experiment ran Isolation Forest, LOF and One-Class SVM on 100, 300 and 900 generated rows, with eight features and three repetitions per size. All 27 runs completed. At 900 rows, median measured detection times were 0.392, 0.020 and 0.049 seconds, respectively, on the recorded macOS/ARM environment. This small in-memory experiment measures neither native-library peak memory nor the scaling of the 284,000-row fraud dataset. It supports reporting the workload and omissions explicitly, but not a general speed ranking.

APPENDIX C
FULL PER-TYPE MATRICES

The tables below give the mean recall of every detector of the registry on each injected type, over five seeds for the synthetic tables and as the median over the 42 real tables

TABLE VIII
SYNTHETIC CUSTOMER TABLE, MEAN RECALL OVER FIVE SEEDS.

Method	Miss.	Dup.	Uni.	Multi.	Drift	Inv.
Z-score	0.00	0.00	0.93	0.00	0.00	0.24
IQR	0.05	0.04	0.99	0.04	0.25	**1.00**
Great Expectations	**1.00**	0.00	0.21	0.00	0.00	0.00
Isolation Forest	0.09	0.11	0.99	0.89	0.33	0.60
LOF	0.28	0.16	0.70	0.21	0.50	0.67
One-Class SVM	0.07	0.10	**1.00**	0.90	0.29	0.54
ECOD	0.10	0.13	0.68	0.52	0.26	0.36
COPOD	0.09	0.13	0.89	0.51	0.34	0.30
HBOS	0.00	0.00	0.90	0.00	0.00	0.32
KNN	0.09	0.10	**1.00**	0.39	0.41	0.81
PCA	0.10	0.12	**1.00**	0.48	0.23	0.58
Autoencoder	0.12	0.10	0.81	0.83	0.29	0.53
VAE	0.07	0.11	0.99	0.92	0.30	0.66
Deep SVDD	0.19	0.18	0.33	0.13	0.16	0.32
DIF	0.09	0.07	**1.00**	**1.00**	0.33	**1.00**
AE1SVM	0.09	0.10	0.99	0.78	0.27	0.61
LUNAR	0.44	0.39	**1.00**	0.63	**0.65**	0.90
LSTM autoencoder	0.08	0.09	0.97	0.68	0.25	0.53
Drift detector	0.00	0.00	0.00	0.00	0.00	0.00
Dedup rule	0.00	1.00	0.00	0.00	0.00	0.00

TABLE IX
SYNTHETIC CARD TRANSACTIONS, MEAN RECALL OVER FIVE SEEDS.

Method	Miss.	Dup.	Uni.	Multi.	Drift	Inv.
Z-score	0.00	0.00	0.63	0.00	0.00	0.26
IQR	0.09	0.14	0.98	0.57	0.37	0.63
Great Expectations	**1.00**	0.00	0.14	0.00	0.00	0.00
Isolation Forest	0.09	0.18	0.97	0.49	0.38	0.56
LOF	0.11	0.15	0.72	0.64	0.30	0.45
One-Class SVM	0.10	0.26	0.89	0.18	0.36	0.39
ECOD	0.10	0.20	0.67	0.45	0.29	0.67
COPOD	0.09	0.20	0.86	0.43	0.33	0.45
HBOS	0.00	0.00	0.69	0.00	0.00	0.26
KNN	0.11	0.17	0.97	0.33	0.39	0.57
PCA	0.08	0.19	0.86	0.16	0.27	0.37
Autoencoder	0.09	0.19	0.84	0.17	0.30	0.46
VAE	0.07	0.24	0.86	0.21	0.34	0.39
Deep SVDD	0.12	0.14	0.74	0.70	0.27	0.54
DIF	0.09	0.20	0.97	0.37	0.36	0.60
AE1SVM	0.09	0.24	0.88	0.16	0.34	0.40
LUNAR	0.22	0.29	**0.99**	**0.73**	**0.55**	**0.75**
Dedup rule	0.00	1.00	0.00	0.00	0.00	0.00

TABLE X
SYNTHETIC SENSOR READINGS, MEAN RECALL OVER FIVE SEEDS.

Method	Miss.	Dup.	Uni.	Multi.	Drift	Inv.
Z-score	0.00	0.00	**1.00**	0.00	0.00	0.26
IQR	0.03	0.05	**1.00**	0.02	0.94	**1.00**
Great Expectations	**1.00**	0.00	0.23	0.00	0.00	0.00
Isolation Forest	0.08	0.12	**1.00**	0.32	0.96	0.97
LOF	0.19	0.18	0.33	0.34	0.40	0.27
One-Class SVM	0.08	0.10	**1.00**	0.33	0.94	**1.00**
ECOD	0.10	0.13	0.81	0.40	0.80	0.58
COPOD	0.07	0.13	0.97	0.45	0.96	0.46
HBOS	0.00	0.00	**1.00**	0.00	0.00	0.26
KNN	0.11	0.10	**1.00**	0.26	**1.00**	**1.00**
PCA	0.09	0.12	**1.00**	0.26	0.63	0.95
Autoencoder	0.07	0.11	0.93	0.62	0.71	0.94
VAE	0.07	0.09	**1.00**	0.25	0.94	**1.00**
Deep SVDD	0.14	0.14	0.82	0.19	0.25	0.59
DIF	0.05	0.06	**1.00**	**0.99**	0.80	**1.00**
AE1SVM	0.08	0.10	**1.00**	0.49	0.80	**1.00**
LUNAR	0.32	0.30	**1.00**	0.41	**1.00**	**1.00**
Dedup rule	0.00	1.00	0.00	0.00	0.00	0.00

TABLE XI
TYPED INJECTION INTO 42 REAL TABLES, MEDIAN RECALL OVER THE TABLES.

Method	Miss.	Dup.	Uni.	Multi.	Drift	Inv.
Z-score	0.00	0.00	0.00	0.00	0.00	0.43
IQR	0.19	0.21	**1.00**	0.39	0.64	**0.93**
Great Expectations	**1.00**	0.00	0.23	0.00	0.00	0.42
Isolation Forest	0.11	0.13	0.61	0.37	0.45	0.59
LOF	0.28	0.13	0.68	0.33	0.34	0.70
One-Class SVM	0.10	0.13	0.75	0.17	0.31	0.73
ECOD	0.10	0.14	0.44	0.25	0.36	0.44
COPOD	0.10	0.15	0.45	0.23	0.48	0.41
HBOS	0.00	0.01	0.04	0.02	0.01	0.32
KNN	0.19	0.10	0.91	0.30	0.46	0.80
PCA	0.12	0.14	0.54	0.19	0.23	0.71
Autoencoder	0.11	0.12	0.76	0.29	0.36	0.76
VAE	0.10	0.13	0.72	0.18	0.27	0.72
Deep SVDD	0.14	0.13	0.72	0.23	0.37	0.70
DIF	0.11	0.12	0.81	0.28	0.39	0.78
AE1SVM	0.11	0.14	0.63	0.17	0.18	0.76
LUNAR	0.38	0.25	0.98	**0.63**	**0.78**	0.90
Drift detector	0.00	0.00	0.00	0.00	0.00	0.00
Dedup rule	0.00	1.00	0.00	0.00	0.00	0.00

for the last one. The best value of each column among the detectors of the registry is in bold.

APPENDIX D
DETAILED RESULTS

This appendix holds the per-dataset tables and the detailed text of the studies summarised in Section IV.

A. Example 1: Credit Card Fraud

Table XII reports the detectors on Credit Card Fraud: the eight that scale to 284k rows, then the six neural ones, which are fitted on at most 20,000 rows. The statistical rules recover most frauds (recall 0.87–0.97) at a precision below 0.04: a fixed per-column rule cannot separate a 0.17% positive class from the tails of legitimate traffic. Great Expectations is the most selective of the rules (3% of rows) and has the best AP of the three (0.182 ± 0.046). Isolation Forest behaves differently now that its contamination matches the prevalence: it flags only 0.17% of rows and reaches a precision of 0.20, the first usable operating point among the rules and Isolation Forest, but its recall drops to 0.21 and both are seed-sensitive ($F_1 = 0.20 \pm 0.07$). The column-wise detectors added to the registry do better than Isolation Forest on this dataset: with the same 0.2% of rows flagged, HBOS reaches the best precision of the table (0.34 ± 0.05), COPOD and ECOD an F_1 of 0.30 and 0.26 against 0.20, and the AP of HBOS and COPOD (0.22 and 0.21) is above that of Great Expectations. PCA is

TABLE XII
DETECTORS ON CREDIT CARD FRAUD, MEAN OVER FIVE SEEDS ON THE 15% TEST SPLIT. HIGHER IS BETTER EXCEPT FLAGGED ROWS AND TIME.

Detector	Prec.	Rec.	F_1	ROC	PR	Flag.%	Time (s)
Z-score	0.011	0.870	0.021	0.926	0.078	13.4	**0.04**
IQR	0.003	**0.966**	0.007	0.911	0.018	48.6	0.12
Great Expectations	0.037	0.676	0.070	0.830	0.182	3.1	1.25
Isolation Forest	0.202	0.206	0.203	0.934	0.120	0.2	1.75
ECOD	0.260	0.268	0.262	0.930	0.175	0.2	3.13
COPOD	0.310	0.299	0.303	0.927	0.213	0.2	3.13
HBOS	**0.337**	0.318	0.326	0.931	0.216	0.2	0.99
PCA	0.139	0.132	0.135	0.831	0.105	0.2	0.72
Autoencoder	0.083	0.141	0.103	0.892	0.066	0.3	–
VAE	0.162	0.237	0.191	0.929	0.142	0.2	–
Deep SVDD	0.261	0.546	**0.346**	0.918	**0.289**	0.4	–
DIF	0.287	0.355	0.317	0.922	0.242	0.2	–
AE1SVM	0.184	0.223	0.201	0.937	0.144	0.2	–
LUNAR	0.138	0.535	0.216	**0.941**	0.136	0.7	–

below Isolation Forest ($F_1 = 0.14 \pm 0.14$). Among the neural models, Deep SVDD has the best F_1 (0.35 ± 0.08) and AP (0.29 ± 0.10) of the table, at a recall of 0.55, and LUNAR the best ROC-AUC (0.941) with a precision of only 0.14 and 0.7% of rows flagged; the autoencoder is the weakest ($F_1 = 0.10$). Seed spreads are large for every learned model, so the order among the better ones is not firm. Great Expectations took 1.3 s on this run, against 0.03–0.12 s for the statistical rules; these times vary between runs, and the neural models were timed under load, so their column is left empty (Section IV-J measures them alone). LOF, KNN and the One-Class SVM were not run at 284k rows, where their cost was expected to be prohibitive; I did not measure it.

B. Example 2: Cardio, rare and distinct anomalies

Table XIII runs all detectors on Cardio, where the anomalies are both rare (9.6%) and distributionally distinct. Here the unsupervised detectors work: Isolation Forest reaches $F_1 = 0.62$, AP 0.70 and ROC-AUC 0.94, and the One-Class SVM is close behind. Z-score also does well on this dataset ($F_1 = 0.61$) because the 21 features are already standardized and the anomalies stand out on several of them at once. ECOD and COPOD reach the level of Isolation Forest (AP 0.69 for both against 0.70, F_1 0.57 and 0.60), while HBOS ($F_1 = 0.51$), KNN (0.43) and PCA (0.36) fall below it. Among the neural models, DIF ($F_1 = 0.59$, AP 0.67) and AE1SVM (0.61, 0.64) come close to Isolation Forest, whereas the autoencoder, Deep SVDD and LUNAR are far below (F_1 between 0.29 and 0.35). This is the counterpoint to Credit Card Fraud and Water Quality: the value of a learned detector shows when the anomalies are genuinely rare outliers, not when the positive class is a 0.17% mixed-defect label or a 61% majority. Isolation Forest and the One-Class SVM carry a wide seed spread on this small dataset (F_1 standard deviations of 0.14 and 0.17).

TABLE XIII
ALL DETECTORS ON CARDIO, MEAN OVER FIVE SEEDS ON THE 15% TEST SPLIT. 1,831 RAW ROWS (1,822 AFTER DEDUPLICATION), 9.6% ANOMALIES.

Detector	Prec.	Rec.	F_1	ROC	PR	Flag.%
Z-score	0.49	0.81	0.61	0.93	0.61	16.6
IQR	0.21	**0.90**	0.33	0.78	0.30	43.4
Great Expectations	0.50	0.21	0.29	0.59	0.19	4.6
Isolation Forest	**0.64**	0.61	**0.62**	0.94	**0.70**	9.9
LOF	0.25	0.27	0.26	0.62	0.24	10.7
One-Class SVM	0.56	0.61	0.58	0.94	0.63	10.7
ECOD	0.57	0.59	0.57	**0.95**	0.69	10.2
COPOD	0.61	0.61	0.60	0.93	0.69	9.9
HBOS	0.52	0.52	0.51	0.86	0.57	10.2
KNN	0.40	0.47	0.43	0.78	0.47	11.7
PCA	0.33	0.39	0.36	0.73	0.37	11.4
Autoencoder	0.32	0.38	0.35	0.74	0.35	11.7
VAE	0.41	0.46	0.43	0.87	0.45	11.2
Deep SVDD	0.26	0.41	0.32	0.69	0.29	15.8
DIF	0.53	0.69	0.59	0.94	0.67	13.1
AE1SVM	0.58	0.64	0.61	0.94	0.64	10.9
LUNAR	0.41	0.39	0.29	0.65	0.33	18.8

TABLE XIV
ALL DETECTORS ON WATER QUALITY, MEAN OVER FIVE SEEDS ON THE 15% TEST SPLIT. TARGET: "NOT POTABLE", BASE RATE 0.61.

Detector	Prec.	Rec.	F_1	ROC	PR	Flag.%
Z-score	0.41	0.02	0.05	0.46	0.58	3.5
IQR	0.57	0.09	0.16	0.49	0.60	10.0
Great Expectations	**0.62**	0.41	0.49	**0.51**	**0.61**	40.1
Isolation Forest	0.57	0.46	0.51	0.45	0.57	49.7
LOF	0.58	0.46	0.52	0.46	0.58	48.7
One-Class SVM	0.57	0.45	0.50	0.46	0.57	48.4
ECOD	0.58	0.47	0.52	0.45	0.57	49.6
COPOD	0.58	0.48	0.52	0.46	0.57	50.3
HBOS	0.58	0.47	0.52	0.46	0.58	49.8
KNN	0.57	0.46	0.51	0.45	0.57	48.4
PCA	0.59	0.48	0.53	0.47	0.59	49.8
Autoencoder	0.60	0.49	0.54	0.47	0.58	50.0
VAE	0.58	0.48	0.53	0.46	0.58	50.7
Deep SVDD	0.61	0.52	**0.56**	0.49	0.60	52.4
DIF	0.58	0.48	0.53	0.46	0.58	51.1
AE1SVM	0.56	0.45	0.50	0.45	0.57	49.3
LUNAR	0.58	**0.53**	0.55	0.45	0.57	55.4

C. Example 3: Water Quality, a common target class

Table XIV runs all detectors, the neural ones included, on Water Quality, where "not potable" is the target and covers 61% of the rows. Every detector's ROC-AUC sits between 0.45 and 0.51 and every AP lies within 0.05 of the 0.61 base rate: the unsupervised detectors are essentially at chance. This is the expected outcome and a useful negative result. When the target class is common and not distributionally distinct from the rest, "anomaly" detection has nothing to key on. The dataset still exercises the pipeline on real missing data: Great Expectations flags 40% of rows on its not-null expectations, but the nulls here are uncorrelated with potability, so that recall buys no precision.

TABLE XV
ALL DETECTORS ON THE SYNTHETIC DATASET, MEAN OVER FIVE SEEDS ON THE 15% TEST SPLIT. TIME IS THE IN-PIPELINE FIT+PREDICT+SCORE LATENCY.

Detector	Prec.	Rec.	F_1	ROC	PR	Time (s)
Z-score	**1.000**	0.196	0.328	0.742	0.544	**0.002**
IQR	0.584	0.396	0.472	0.675	0.452	0.003
Great Expectations	0.951	0.202	0.333	0.600	0.333	0.291
Isolation Forest	0.495	0.499	0.497	0.735	0.485	0.176
LOF	0.388	0.420	0.404	0.659	0.437	0.141
One-Class SVM	0.478	0.485	0.481	0.722	0.520	2.696
ECOD	0.341	0.341	0.341	0.675	0.283	0.019
COPOD	0.367	0.375	0.371	0.682	0.302	0.018
HBOS	**1.000**	0.204	0.339	0.602	0.343	0.197
KNN	0.466	0.469	0.467	0.745	0.520	0.083
PCA	0.405	0.417	0.411	0.689	0.485	0.007
Autoencoder	0.438	0.445	0.441	0.694	0.455	–
VAE	0.502	0.507	0.504	0.742	0.548	–
Deep SVDD	0.221	0.219	0.219	0.573	0.221	–
DIF	0.589	0.582	**0.585**	**0.772**	**0.648**	–
AE1SVM	0.452	0.467	0.459	0.719	0.502	–
LUNAR	0.345	**0.668**	0.410	0.728	0.495	–

D. Synthetic dataset: aggregate metrics

Table XV reports the seventeen detectors on the synthetic dataset. Aggregate metrics still compress the families together: F_1 ranges from 0.22 (Deep SVDD) to 0.59 (DIF) and AP from 0.22 to 0.65. Among the earlier detectors Isolation Forest has the best F_1 (0.50); Z-score and HBOS both reach a perfect precision by firing only on the most extreme rows, at a recall of 0.20. DIF has the best F_1, ROC-AUC and AP, the VAE matches Isolation Forest, and LUNAR has the best recall (0.67) with a low precision, which fits the share of clean rows it flags in the per-type matrix. Read alone, these numbers suggest the detector choice barely matters among the middle of the table; the per-type breakdown shows the opposite. The neural times are left empty because those runs shared the machine, and Section IV-J gives their cost.

E. Two further synthetic schemas

The synthetic matrix comes from one table of customers, so its pattern could come from that table. Section III-H describes the two further generators and their four hypotheses. The customer table and its published results are untouched; its matrix below is the published one.

The structural rows carry over exactly: Great Expectations finds every missing value and the deduplication rule every duplicate, on all three schemas. The overall profile is stable too: the rank correlation between the method-by-type matrices of two schemas is 0.93 on average (0.91 to 0.96) for the detectors of the first six rows, and 0.89 (0.86 to 0.92) when the new classical and the neural detectors are added. Two things do not carry over. For multivariate outliers, Isolation Forest and One-Class SVM reach 0.89 and 0.90 on the customers, but only 0.49 and 0.18 on the transactions and 0.32 and 0.33 on the sensors, and on the transactions the IQR rule reaches 0.57. For drift, the best recall is 0.50 on the customers and 0.38 on the transactions but 0.96 on the sensors, where a factor of 1.5 on a signal of about -70 dBm moves values several standard deviations away. The per-row drift detector of the registry, which was designed for dilute shifts, flags nothing on any schema. Two of the four hypotheses (structural rows, rank stability) hold; the multivariate gap and the weakness of drift do not. The new detectors do not remove this dependence. On multivariate outliers the best model is not the same on each schema: DIF reaches 1.00 on the customers and 0.99 on the sensors but only 0.37 on the transactions, where LUNAR (0.73) and Deep SVDD (0.70) do best. On drift, LUNAR and KNN reach 1.00 on the sensors, and LUNAR 0.65 and 0.55 on the customers and the transactions. The injected magnitudes are set in relative terms, so how visible drift and multivariate outliers are depends on how the column is scaled and correlated, not only on the type. The three schemas share an author and an injection logic, so this shows that the profile does not depend on the shape of one table, not that it holds on real data (Section IV-F).

TABLE XVI
MEAN RECALL OVER FIVE SEEDS ON THE TWO FURTHER SCHEMAS FOR THE FIRST SIX METHODS THAT CARRY THE PATTERN (CUSTOMER TABLE IN FIGURE 1; THE FULL MATRICES WITH THE OTHER DETECTORS ARE IN APPENDIX C). COLUMNS: MISSING VALUES, DUPLICATES, UNIVARIATE OUTLIERS, MULTIVARIATE OUTLIERS, DRIFT, INVALID FORMATS.

Schema	Method	Miss.	Dup.	Uni.	Multi.	Drift	Inv.
Transactions	Great Expect.	1.00	0.00	0.14	0.00	0.00	0.00
	Dedup rule	0.00	1.00	0.00	0.00	0.00	0.00
	IQR	0.09	0.14	0.98	0.57	0.37	0.63
	Isolation Forest	0.09	0.18	0.97	0.49	0.38	0.56
	One-Class SVM	0.10	0.26	0.89	0.18	0.36	0.39
Sensors	Great Expect.	1.00	0.00	0.23	0.00	0.00	0.00
	Dedup rule	0.00	1.00	0.00	0.00	0.00	0.00
	IQR	0.03	0.05	1.00	0.02	0.94	1.00
	Isolation Forest	0.08	0.12	1.00	0.32	0.96	0.97
	One-Class SVM	0.08	0.10	1.00	0.33	0.94	1.00

F. Neural models and further classical detectors in the per-type matrix

The first six rows of Figure 1 have no learned reconstruction model. I added two on the same synthetic data, splits and seeds (protocol in Section III-H): the sklearn autoencoder of the registry, and the LSTM autoencoder of Appendix D-J. All ten trainings stopped before the cap (60 to 178 epochs), and the other rows, recomputed by the same code, equal the published matrix.

Those two rows do not change the picture. The autoencoder detects multivariate outliers (0.83 ± 0.10) about as well as Isolation Forest (0.89) and One-Class SVM (0.90), as expected for a model that reconstructs correlations, and the LSTM is a little lower (0.68 ± 0.12). Both are close to the learned detectors on the structural types, and on missing values and duplicates they are no better than the best row-level detector. The hypothesis I expected to matter, that the sequence model would catch temporal drift better than every row-level detector, is not supported: its recall is 0.25 ± 0.07 against 0.50 for LOF, 0.33 for Isolation Forest and 0.29 for the autoencoder. Three

of the four hypotheses hold (structural types, convergence, multivariate outliers). The synthetic customers are independent rows, so the signup order carries a sequence only for the drifted rows, which are the most recent ones; a window of ten rows that are mostly clean and have shifted values in only some of the columns gives the LSTM little to learn from. This says that this construction of drift is not a sequence problem, not that sequence models cannot detect drift. With five seeds the standard deviations are indicative and none of these comparisons is a test.

The complete-registry study then added five classical detectors and five neural ones (Section III) to the same matrix, with three hypotheses on it written beforehand (`verification/full_registry.md`). None is supported. The detectors that read one column at a time were expected to miss multivariate outliers (recall at most 0.20 for ECOD, COPOD and HBOS, and Isolation Forest at least 0.50 above them): only HBOS does (0.00), while ECOD and COPOD reach 0.52 and 0.51. PCA was expected to match the autoencoder within ten points, since both rebuild the rows, and it does not (0.48 against 0.83): a reconstruction through 95% of the variance rebuilds a row that breaks the correlations only when the broken direction is a minor one. No new detector was expected to beat the best earlier row-level detector on drift by more than ten points; LUNAR does (0.65 against 0.50 for LOF). Its other rows are also the highest of the matrix on missing values (0.44) and duplicates (0.39), but it flags 41% of the clean rows, against 9% to 16% for the other neural models, so its recall comes with a much higher cost in false alarms; the same holds on the real tables of Section IV-F. DIF reaches 1.00 on both univariate and multivariate outliers and on invalid formats, with 9% of clean rows flagged; the VAE (0.92) and AE1SVM (0.78) follow the autoencoder on multivariate outliers; Deep SVDD is the weakest neural model on univariate and multivariate outliers, drift and invalid formats.

G. Tables of the whole-pool extension

TABLE XVII
MEDIAN OVER THE 100 LABELLED PROBLEMS OF THE F_1 AND OF THE SHARE OF FLAGGED ROWS (%), AT THE TRAINING-PREVALENCE THRESHOLD (PREV.) AND AT A CONSTANT CONTAMINATION OF 0.02 (FIXED).

Detector	F_1 prev.	F_1 fixed	Flag.% prev.	Flag.% fixed
Z-score	0.164	0.164	11.4	11.4
IQR	0.160	0.160	22.8	22.8
Great Expectations	0.052	0.052	1.6	1.6
Isolation Forest	0.189	0.068	10.0	2.1
LOF	0.163	0.052	10.1	2.0
One-Class SVM	0.214	0.072	10.6	2.4
ECOD	0.214	0.068	10.0	2.1
COPOD	0.199	0.073	10.0	2.2
HBOS	0.213	0.067	10.0	2.1
KNN	0.202	0.074	10.0	2.1
PCA	0.169	0.068	11.4	2.4
Autoencoder	0.190	0.100	12.4	3.0
VAE	0.184	0.069	10.0	2.2
Deep SVDD	0.193	0.101	14.0	4.3
DIF	0.217	0.120	13.6	2.9
AE1SVM	0.184	0.054	10.3	2.2
LUNAR	0.131	0.064	9.1	2.3

TABLE XVIII
ISOLATION FOREST ON THE 100 LABELLED PROBLEMS: MEDIAN PRECISION, RECALL, F_1 AND SHARE OF FLAGGED ROWS (%) OVER THE CONTAMINATION GRID.

Contamination	Prec.	Rec.	F_1	Flag.%
0.001	0.062	0.001	0.003	0.1
0.010	0.125	0.018	0.032	1.1
0.050	0.155	0.117	0.114	5.2
0.100	0.141	0.213	0.166	10.2
0.200	0.148	0.377	0.203	20.3

H. Chronological pilot on Credit Card

The results above use a random stratified split, and Credit Card is the only example with a time variable. The protocol of Section III-H fits the detectors on the oldest 70% of the rows and scores the newest 15%, with a stratified control. The detectors are fitted on the first $198{,}609$ rows (366 frauds) and scored on the last $42{,}557$ rows (52 frauds, 0.12%). The detectors are the four of the pilot plus ECOD, COPOD, HBOS and PCA, which scale to this size; the rerun reproduces the first four rows exactly. The pilot is a separate experiment and does not replace the benchmark.

Average precision is lower under the chronological split for all four detectors, but the interval excludes the stratified mean only for Great Expectations and Isolation Forest; for the Z-score and the IQR rule the difference is within the uncertainty of 52 test frauds. This exclusion criterion was fixed in advance as an exploratory check. It is not a confidence interval or a test for the difference between the two protocols: the bootstrap reflects only the chronological arm, the stratified mean is itself uncertain (five seeds), and the share of frauds changes between partitions (0.18% in training, 0.12% in the test period), so part of the gap may come from prevalence alone, which this pilot does not separate from the effect of time. The best detector also changes: Great Expectations leads the stratified control, while the Z-score has the highest chronological point estimate, with overlapping intervals. Isolation Forest keeps a high ROC-AUC (0.936) but flags only 0.08% of the future rows and none of the 52 frauds: the threshold set from training scores does not carry over to the later period. This is a single two-day dataset with few positives, so it indicates a direction and an order of magnitude, not a general rule.

The four detectors added later behave differently from one another. ECOD and COPOD lose part of their stratified average precision (0.175 to 0.075 and 0.212 to 0.127) and, like Isolation Forest, flag under 0.1% of the future rows; they find one fraud and three of the 52 (recall 0.02 and 0.06). HBOS keeps the highest chronological average precision of the table (0.151, interval $[0.058, 0.268]$ overlapping the others'), with a precision of 0.26 and a recall of 0.19 on 0.09% of the rows: the first detector of this study whose training-prevalence threshold still catches frauds in the later period. PCA collapses (0.002,

TABLE XIX
MEDIAN RECALL ON MULTIVARIATE OUTLIERS OVER THE ELIGIBLE TABLES, BY SEVERITY MULTIPLIER OF THE INJECTED AMPLITUDE.

Detector	×0.5	×1	×2	×4
Z-score	0.00	0.00	0.00	0.00
IQR	0.22	0.38	0.53	0.62
Great Expectations	0.00	0.00	0.00	0.00
Isolation Forest	0.24	0.33	0.36	0.40
LOF	0.33	0.30	0.32	0.32
One-Class SVM	0.16	0.17	0.18	0.20
ECOD	0.21	0.24	0.27	0.30
COPOD	0.20	0.22	0.25	0.28
HBOS	0.00	0.00	0.00	0.00
KNN	0.31	0.32	0.34	0.35
PCA	0.18	0.18	0.21	0.19
Autoencoder	0.25	0.27	0.30	0.29
VAE	0.18	0.19	0.21	0.19
Deep SVDD	0.22	0.23	0.23	0.23
DIF	0.25	0.35	0.36	0.44
AE1SVM	0.16	0.18	0.17	0.18
LUNAR	0.60	0.66	0.66	0.68

TABLE XX
CHRONOLOGICAL SPLIT ON THE POOL DATASETS WITH A TIME VARIABLE: ANOMALIES IN THE TEST PARTITION, PREVALENCE IN TRAINING AND TEST (%), AND THE NUMBER OF DETECTORS WHOSE AP IS LOWER THAN UNDER THE STRATIFIED SPLIT.

Dataset	Test anom.	Train %	Test %	Lower
Credit Card	52	0.2	0.1	12 of 14
Bank Marketing	2668	5.8	39.3	0 of 16
Occupancy Detection	405	11.0	15.4	11 of 17
Online Shoppers	144	8.9	8.3	8 of 17
Room Occupancy	excluded: 0 test anomalies			

ROC-AUC 0.61), with no fraud found. The intervals are wide, so this shows that the column-wise detectors are not all alike under a time split, not that HBOS is better.

To check the window monitor with a known change, a mean shift of δ training standard deviations was added to five features from the middle of the test period. The alert threshold was fixed from the validation period before the test (the largest validation window score, 0.91). Shifts of 1 and 2 standard deviations were detected in the second window after the change (573 seconds), and a shift of 0.5 was not detected; no window before the change raised an alert (22 windows). Credit Card has no labelled change in distribution, so I make no claim about its natural drift.

I. Decisions under temporal shift

The pilot above leaves two questions open: whether a threshold chosen on the past can be repaired without labels, and how much of the gap is only a change in prevalence. The study of Section III-H runs on Credit Card and on Bank Marketing. The Credit Card results were already known, so it is an independent test only for Bank Marketing.

On Bank Marketing the prevalence moves a lot: 5.8% in training, 11.5% in validation and 39.3% in the test period (Credit Card: 0.18%, 0.13% and 0.12%). The average precision under the chronological split is higher than under the stratified one for every detector (0.55 against 0.17 for the Z-score, 0.42 against 0.19 for Isolation Forest), but the ROC-AUC of the chronological split is between 0.48 and 0.63, close to chance. Matching the control to the test prevalence brings its average precision to 0.40–0.52, so the gap, a median of 0.26, falls to 0.06: on this dataset the apparent gain is a change in prevalence, not in detection. On Credit Card the same matching reduces the median gap from 0.053 to 0.031, which is less than half of it, so a smaller part of the gap is explained by prevalence there.

TABLE XXI
AVERAGE PRECISION (PR) ON THE TEST SPLIT, STRATIFIED (STRAT.) CONTROL VERSUS CHRONOLOGICAL (CHRON.) SPLIT WITH ITS 95% BLOCK-BOOTSTRAP INTERVAL, ROC-AUC OF THE CHRONOLOGICAL SPLIT, AND THE SHARE OF FUTURE ROWS FLAGGED AT THE TRAINING-PREVALENCE THRESHOLD.

Detector	PR, strat.	PR, chron.	ROC, chron.	Flag.%
Z-score	0.078 ± 0.015	0.048 [0.020, 0.087]	0.935	12.79
IQR	0.018 ± 0.003	0.013 [0.006, 0.023]	0.917	47.10
Great Expectations	0.183 ± 0.046	0.045 [0.013, 0.088]	0.758	3.45
Isolation Forest	0.110 ± 0.039	0.035 [0.016, 0.066]	0.936	0.08
ECOD	0.175 ± 0.035	0.075 [0.027, 0.146]	0.936	0.08
COPOD	0.212 ± 0.046	0.127 [0.045, 0.216]	0.936	0.07
HBOS	0.207 ± 0.038	0.151 [0.058, 0.268]	0.928	0.09
PCA	0.108 ± 0.117	0.002 [0.001, 0.003]	0.608	0.43

The ranking of the detectors is not stable. The Spearman correlation between the two orderings by average precision is 0.80 on Bank Marketing and 0.40 on Credit Card, below the 0.8 expected on both. The three thresholds do not behave as I expected. On Bank Marketing the share of rows flagged moves closer to the training prevalence from P0 to P1 to P2 (median error $|\ln(\text{share}/\pi)|$ of 1.51, 1.02 and 0.66), but on Credit Card P1 is no better than P0 (0.78 against 0.73), so the ordering holds on one dataset only. The reason is clearer in the decisions. On Bank Marketing, P0 flags 41% of the test rows with the Z-score and 37% with Isolation Forest, because the later rows score higher than the training rows, which gives an F_1 of 0.51 and 0.41 by accident of a prevalence of 39%; the rolling threshold targets a share of 5.8%, flags 1.7% and 6.3% of the rows and drops the F_1 to 0.02 and 0.11. A label-free recalibration that keeps the training prevalence cannot follow a prevalence that changes. On Credit Card the opposite holds: no detector detects any of the 52 frauds with P0 ($F_1 = 0$), while the rolling threshold gives a recall of 0.12 for Great Expectations and 0.14 for the Z-score, at a precision below 0.08. The F_1 of P2 is at least that of P0 for 5 of the 8 pairs, which satisfies the hypothesis, but four of the five are Credit Card pairs where P0 is zero. Overall one hypothesis of four holds. This study does not establish a rule for thresholds under shift: it shows that a threshold fixed on the past can fail in both directions, that recalibrating without labels needs an estimate of the prevalence, and that part of what a chronological split changes is only the prevalence.

J. LSTM autoencoder pilot

A last pilot asks whether the preceding rows of the transaction stream carry a fraud signal that a row-level autoencoder cannot see (protocol in Section III-H; the architecture was not tuned).

As a check before reading the real data, a synthetic stream whose anomalies break the continuity of an autoregressive signal was scored by both models: the LSTM reaches an average precision of 0.875 against 0.006 for the row-level autoencoder. On Credit Card, over five initialisation seeds, the LSTM reaches an average precision of 0.037 ± 0.006 on the 52 test frauds against 0.024 ± 0.010 for the row-level autoencoder (ROC-AUC 0.926 against 0.841). The paired block-bootstrap interval of the difference excludes zero for the main seed ($+0.021$, $[+0.005, +0.044]$) but not for the other four, where the difference is positive for three seeds and negative for one. This criterion was set in advance as an exploratory check, not a formal test. The LSTM is comparable to Isolation Forest (0.035) and Great Expectations (0.045) on the same rows, not clearly better. None of the five trainings converged within the budget: the validation loss was still decreasing at the 20th epoch. As for the other detectors, the threshold taken from training scores flags almost none of the frauds in the later period (recall at most 0.02). The result is therefore an indication from a small, budget-limited experiment on one dataset, not a ranking.

REFERENCES

[1] V. Khatri and C. V. Brown, "Designing data governance," *Communications of the ACM*, vol. 53, no. 1, pp. 148–152, 2010.

[2] Y. Zhou, F. Tu, K. Sha, J. Ding, and H. Chen, "A survey on data quality dimensions and tools for machine learning," *arXiv preprint arXiv:2406.19614*, 2024. [Online]. Available: https://arxiv.org/abs/2406.19614

[3] R. Gopa, "AI-augmented ETL pipelines for automated data quality, anomaly detection and governance," *International Journal of Computational and Experimental Science and Engineering (IJCESEN)*, vol. 11, no. 4, pp. 7920–7927, 2025.

[4] D. Marupaka and S. Rangineni, "Machine learning-driven predictive data quality assessment in ETL frameworks," *International Journal of Computer Trends and Technology (IJCTT)*, vol. 72, no. 3, pp. 53–60, 2024.

[5] M. R. Sokkula and S. K. Vuppala, "Implementing machine learning for ETL data transformation and anomaly detection," *International Journal for Multidisciplinary Research (IJFMR)*, vol. 6, no. 6, 2024.

[6] S. Han, X. Hu, H. Huang, M. Jiang, and Y. Zhao, "ADBench: Anomaly detection benchmark," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. [Online]. Available: https://openreview.net/pdf?id=foA_SFQ9zo0

[7] Y. Zhao, Z. Nasrullah, and Z. Li, "PyOD: A python toolbox for scalable outlier detection," *Journal of Machine Learning Research*, vol. 20, no. 96, pp. 1–7, 2019. [Online]. Available: https://jmlr.org/papers/v20/19-011.html

[8] M. Goldstein and S. Uchida, "A comparative evaluation of unsupervised anomaly detection algorithms for multivariate data," *PLoS ONE*, vol. 11, no. 4, p. e0152173, 2016.

[9] Great Expectations, "Great expectations," Open-source data validation library. [Online]. Available: https://greatexpectations.io

[10] A. Mbata, Y. Sripada, and M. Zhong, "A survey of pipeline tools for data engineering," *arXiv preprint arXiv:2406.08335*, 2024. [Online]. Available: https://arxiv.org/abs/2406.08335

[11] S. Schelter, D. Lange, P. Schmidt, M. Celikel, F. Biessmann, and A. Grafberger, "Automating large-scale data quality verification," *Proceedings of the VLDB Endowment*, vol. 11, no. 12, pp. 1781–1794, 2018.

[12] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *2008 Eighth IEEE International Conference on Data Mining*, 2008, pp. 413–422.

[13] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," *ACM SIGMOD Record*, vol. 29, no. 2, pp. 93–104, 2000.

[14] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the support of a high-dimensional distribution," *Neural Computation*, vol. 13, no. 7, pp. 1443–1471, 2001.

[15] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, 2009.

[16] P. Malhotra, A. Ramakrishnan, G. Anand, L. Vig, P. Agarwal, and G. Shroff, "LSTM-based encoder-decoder for multi-sensor anomaly detection," 2016, iCML 2016 Anomaly Detection Workshop.

[17] E. F. Agyemang, "Anomaly detection using unsupervised machine learning algorithms: A simulation study," *Scientific African*, vol. 26, p. e02386, 2024.

[18] M. Dragoi, E. Burceanu, E. Haller, A. Manolache, and F. Brad, "AnoShift: A distribution shift benchmark for unsupervised anomaly detection," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2022.

[19] A. Anwar, B. Moser, D. Herurkar, F. Raue, V. Hegiste, T. Legler, and A. Dengel, "FedAD-Bench: A unified benchmark for federated unsupervised anomaly detection in tabular data," 2024.

[20] S. H. Sun, A. Sankararaman, and B. M. Narayanaswamy, "Online adaptive anomaly thresholding with confidence sequences," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235, 2024, pp. 47 105–47 132. [Online]. Available: https://proceedings.mlr.press/v235/sun24h.html

[21] Z. Li, Y. Zhao, N. Botta, C. Ionescu, and X. Hu, "COPOD: Copula-based outlier detection," in *2020 IEEE International Conference on Data Mining (ICDM)*, 2020, pp. 1118–1123.

[22] Z. Li, Y. Zhao, X. Hu, N. Botta, C. Ionescu, and G. H. Chen, "ECOD: Unsupervised outlier detection using empirical cumulative distribution functions," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 12, pp. 12 181–12 193, 2023.

[23] M. Goldstein and A. Dengel, "Histogram-based outlier score (HBOS): A fast unsupervised anomaly detection algorithm," in *KI-2012: Poster and Demo Track*, 2012, pp. 59–63.

[24] S. Ramaswamy, R. Rastogi, and K. Shim, "Efficient algorithms for mining outliers from large data sets," in *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, 2000, pp. 427–438.

[25] M.-L. Shyu, S.-C. Chen, K. Sarinnapakorn, and L. Chang, "Principal component-based anomaly detection scheme," in *Foundations and Novel Approaches in Data Mining*, ser. Studies in Computational Intelligence. Springer, 2006, pp. 311–329.

[26] S. Chen, Z. Qian, W. Siu, X. Hu, J. Li, S. Li, Y. Qin, T. Yang, Z. Xiao, W. Ye, Y. Zhang, Y. Dong, and Y. Zhao, "PyOD 2: A Python library for outlier detection with LLM-powered model selection," in *Companion Proceedings of the ACM on Web Conference 2025*, 2025, pp. 2807–2810.

[27] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," in *2nd International Conference on Learning Representations (ICLR)*, 2014, arXiv:1312.6114.

[28] L. Ruff, R. Vandermeulen, N. Goernitz, L. Deecke, S. A. Siddiqui, A. Binder, E. Müller, and M. Kloft, "Deep one-class classification," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 80, 2018, pp. 4393–4402.

[29] H. Xu, G. Pang, Y. Wang, and Y. Wang, "Deep isolation forest for anomaly detection," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 12, pp. 12 591–12 604, 2023.

[30] M.-N. Nguyen and N. A. Vien, "Scalable and interpretable one-class SVMs with deep learning and random Fourier features," in *Machine Learning and Knowledge Discovery in Databases*, ser. Lecture Notes in Computer Science. Springer, 2019, pp. 157–172.

[31] A. Goodge, B. Hooi, S.-K. Ng, and W. S. Ng, "LUNAR: Unifying local outlier detection methods via graph neural networks," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, no. 6, 2022, pp. 6737–6745.

[32] Machine Learning Group – ULB, "Credit card fraud detection," Kaggle dataset. [Online]. Available: https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud

[33] D. Campos and J. Bernardes, “Cardiotocography,” UCI Machine Learning Repository, 2000, names as listed by the repository.
[34] A. Kadiwal, “Water quality,” Kaggle dataset, 2021. [Online]. Available: https://www.kaggle.com/datasets/adityakadiwal/water-potability